

NeetCode 250 — First 100 (Roadmap Order)

Self-contained reference. Each problem: statement, sample I/O, simple solution, a simpler/alternative approach, and a short dry run.

SECTION 1: Arrays & Hashing

1. Contains Duplicate

Problem: Return true if any value appears at least twice in the array.

Sample I/O: `[1, 2, 3, 1] → True` | `[1, 2, 3, 4] → False`

```
def hasDuplicate(nums):  
    seen = set()  
    for n in nums:  
        if n in seen:  
            return True  
        seen.add(n)  
    return False
```

Alternative: `return len(nums) != len(set(nums))` **Dry run:** seen builds `{1, 2, 3}`, hits 1 again → `True`.

2. Valid Anagram

Problem: Return true if `t` is an anagram of `s`. **Sample I/O:**

`s="anagram", t="nagaram" → True`

```
def isAnagram(s, t):
    if len(s) != len(t): return False
    count = {}
    for c in s: count[c] = count.get(c, 0) + 1
    for c in t: count[c] = count.get(c, 0) - 1
    return all(v == 0 for v in count.values())
```

Alternative: `return sorted(s) == sorted(t)` **Dry run:** counts cancel to all-zero for true anagrams; leftover nonzero → `False`.

3. Two Sum

Problem: Return indices of two numbers adding to target. **Sample I/O:**

`nums=[2,7,11,15], target=9 → [0,1]`

```
def twoSum(nums, target):
    seen = {}
    for i, n in enumerate(nums):
        if target - n in seen:
            return [seen[target - n], i]
        seen[n] = i
```

Alternative (brute force):

```
def twoSum(nums, target):
    for i in range(len(nums)):
        for j in range(i+1, len(nums)):
            if nums[i]+nums[j] == target: return [i, j]
```

Dry run: at `i=1` (`n=7`), complement 2 already in `seen={2:0}` → return `[0,1]`.

4. Group Anagrams

Problem: Group strings that are anagrams of each other. **Sample I/O:**

["eat", "tea", "tan", "ate", "nat", "bat"] →
[["eat", "tea", "ate"], ["tan", "nat"], ["bat"]]

```
def groupAnagrams(strs):
    groups = {}
    for s in strs:
        key = ''.join(sorted(s))
        groups.setdefault(key, []).append(s)
    return list(groups.values())
```

Alternative (char-count key, avoids $O(k \log k)$ sort):

```
def groupAnagrams(strs):
    groups = {}
    for s in strs:
        count = [0]*26
        for c in s: count[ord(c)-97] += 1
        groups.setdefault(tuple(count), []).append(s)
    return list(groups.values())
```

Dry run: "eat" and "tea" both sort to "aet" → same bucket.

5. Top K Frequent Elements

Problem: Return the k most frequent elements. **Sample I/O:**

nums=[1,1,1,2,2,3], k=2 → [1,2]

```
def topKFrequent(nums, k):
    count = {}
    for n in nums: count[n] = count.get(n, 0) + 1
    freq = [[] for _ in range(len(nums)+1)]
    for n, c in count.items(): freq[c].append(n)
    res = []
```

```

for i in range(len(freq)-1, 0, -1):
    for n in freq[i]:
        res.append(n)
    if len(res) == k: return res

```

Alternative (heap):

```

import heapq
def topKFrequent(nums, k):
    count = {}
    for n in nums: count[n] = count.get(n, 0) + 1
    return heapq.nlargest(k, count.keys(), key=count.get)

```

Dry run: counts {1:3, 2:2, 3:1} → bucket sort gives index 3 → [1], index 2 → [2]; reading from high freq: 1 then 2 → [1, 2].

6. Encode and Decode Strings

Problem: Encode a list of strings into one string and decode it back.

Sample I/O: ["neet", "code", "love", "you"] → encoded → decoded back to same list.

```

def encode(strs):
    return ''.join(f"{len(s)}#{s}" for s in strs)

def decode(s):
    res, i = [], 0
    while i < len(s):
        j = i
        while s[j] != '#': j += 1
        length = int(s[i:j])
        res.append(s[j+1:j+1+length])
        i = j+1+length
    return res

```

Alternative: use a rare delimiter (e.g. `\x00`) if strings guaranteed not to

contain it — simpler but less robust: `'\x00'.join(strs) / s.split('\x00')`. **Dry run:** "neet" → "4#neet"; decode reads 4, takes next 4 chars "neet", advances pointer.

7. Product of Array Except Self

Problem: Return array where each element is product of all others (no division). **Sample I/O:** [1, 2, 3, 4] → [24, 12, 8, 6]

```
def productExceptSelf(nums):
    n = len(nums)
    res = [1]*n
    prefix = 1
    for i in range(n):
        res[i] = prefix
        prefix *= nums[i]
    postfix = 1
    for i in range(n-1, -1, -1):
        res[i] *= postfix
        postfix *= nums[i]
    return res
```

Alternative (uses division, fails on zeros):

```
def productExceptSelf(nums):
    total = 1
    zeros = nums.count(0)
    for n in nums:
        if n != 0: total *= n
    return [0 if zeros>1 else (total if n==0 else total//
```

Dry run: prefix pass gives [1, 1, 2, 6]; postfix pass multiplies in reverse giving [24, 12, 8, 6].

8. Valid Sudoku

Problem: Determine if a 9x9 Sudoku board is valid (rows, cols, 3x3 boxes have no repeats among filled cells). **Sample I/O:** partially filled valid board → True

```
def isValidSudoku(board):
    rows, cols, boxes = {}, {}, {}
    for r in range(9):
        for c in range(9):
            v = board[r][c]
            if v == '.': continue
            b = (r//3, c//3)
            if v in rows.get(r, set()) or v in cols.get(c, set()) or v in boxes.get(b, set()):
                return False
            rows.setdefault(r, set()).add(v)
            cols.setdefault(c, set()).add(v)
            boxes.setdefault(b, set()).add(v)
    return True
```

Alternative: use `defaultdict(set)` instead of `.setdefault` calls for cleaner code (same logic). **Dry run:** duplicate 5 in same row triggers `v in rows.get(r, ...)` → False immediately.

9. Longest Consecutive Sequence

Problem: Return length of longest run of consecutive integers (unsorted input), in O(n). **Sample I/O:** [100, 4, 200, 1, 3, 2] → 4 (sequence 1, 2, 3, 4)

```
def longestConsecutive(nums):
    numSet = set(nums)
    longest = 0
    for n in numSet:
        if n-1 not in numSet: # start of a sequence
            length = 1
            while n+length in numSet:
                length += 1
            longest = max(longest, length)
```

```
return longest
```

Alternative (sort-based, $O(n \log n)$ but simpler to reason about):

```
def longestConsecutive(nums):
    if not nums: return 0
    nums = sorted(set(nums))
    longest = cur = 1
    for i in range(1, len(nums)):
        if nums[i] == nums[i-1]+1: cur += 1
        else: longest, cur = max(longest, cur), 1
    return max(longest, cur)
```

Dry run: 1 has no 0 before it → start; count 1, 2, 3, 4 while each +1 exists → length 4.

SECTION 2: Two Pointers

10. Valid Palindrome

Problem: Check if a string is a palindrome, ignoring non-alphanumeric chars and case. **Sample I/O:** "A man, a plan, a canal:

Panama" → True

```
def isPalindrome(s):
    l, r = 0, len(s)-1
    while l < r:
        while l < r and not s[l].isalnum(): l += 1
        while l < r and not s[r].isalnum(): r -= 1
        if s[l].lower() != s[r].lower(): return False
        l += 1; r -= 1
    return True
```

Alternative (simpler, more memory):

```
def isPalindrome(s):
    clean = [c.lower() for c in s if c.isalnum()]
    return clean == clean[::-1]
```

Dry run: pointers skip spaces/punctuation, compare 'a'=='a', 'm'=='m'... converge without mismatch → True.

11. Two Sum II (Sorted Input)

Problem: Sorted array; return 1-indexed pair summing to target.

Sample I/O: numbers=[2,7,11,15], target=9 → [1,2]

```
def twoSum(numbers, target):
    l, r = 0, len(numbers)-1
    while l < r:
        s = numbers[l] + numbers[r]
        if s == target: return [l+1, r+1]
        elif s < target: l += 1
        else: r -= 1
```

Alternative (hash map, doesn't need sorted-ness but uses O(n) space):

```
def twoSum(numbers, target):
    seen = {}
    for i, n in enumerate(numbers):
        if target-n in seen: return [seen[target-n]+1, i+1]
        seen[n] = i
```

Dry run: 2+15=17>9 → move r left; 2+11=13>9 → r left; 2+7=9 → return [1,2].

12. 3Sum

Problem: Find all unique triplets summing to 0. **Sample I/O:**

`[-1, 0, 1, 2, -1, -4] → [[-1, -1, 2], [-1, 0, 1]]`

```
def threeSum(nums):
    nums.sort()
    res = []
    for i in range(len(nums)):
        if i > 0 and nums[i] == nums[i-1]: continue
        l, r = i+1, len(nums)-1
        while l < r:
            total = nums[i]+nums[l]+nums[r]
            if total < 0: l += 1
            elif total > 0: r -= 1
            else:
                res.append([nums[i], nums[l], nums[r]])
                l += 1
                while nums[l] == nums[l-1] and l < r: l += 1
    return res
```

Alternative (brute force with set dedup, $O(n^2)$ space):

```
def threeSum(nums):
    nums.sort()
    res = set()
    for i in range(len(nums)):
        seen = set()
        for j in range(i+1, len(nums)):
            need = -nums[i]-nums[j]
            if need in seen: res.add(tuple(sorted((nums[i],
                                                    nums[j], need))))
            seen.add(nums[j])
    return [list(t) for t in res]
```

Dry run: sorted `[-4, -1, -1, 0, 1, 2]`; fix `i=1(-1)`, two-pointer finds `-1+0+1=0` and `-1+...` → both valid triplets collected, duplicates skipped via `i>0` and inner-skip checks.

13. Container With Most Water

Problem: Maximize area between two lines (index distance $\times$ min height). **Sample I/O:** [1, 8, 6, 2, 5, 4, 8, 3, 7] $\rightarrow$ 49

```
def maxArea(height):
    l, r = 0, len(height)-1
    best = 0
    while l < r:
        best = max(best, (r-l) * min(height[l], height[r])
        if height[l] < height[r]: l += 1
        else: r -= 1
    return best
```

Alternative (brute force $O(n^2)$):

```
def maxArea(height):
    best = 0
    for i in range(len(height)):
        for j in range(i+1, len(height)):
            best = max(best, (j-i)*min(height[i], height[j])
    return best
```

Dry run: widest span (0, 8) gives $8 \times \min(1, 7) = 8$; moving shorter pointer inward, (1, 6) gives $5 \times 8 = \dots$ eventually best 49 at (1, 8) heights 8 and 7, width 7 $\rightarrow 7 \times 7 = 49$.

14. Trapping Rain Water

Problem: Compute total trapped water given elevation heights. **Sample I/O:** [0, 1, 0, 2, 1, 0, 1, 3, 2, 1, 2, 1] $\rightarrow$ 6

```
def trap(height):
    l, r = 0, len(height)-1
    leftMax, rightMax = height[l], height[r]
    water = 0
    while l < r:
        if leftMax < rightMax:
```

```

        l += 1
        leftMax = max(leftMax, height[l])
        water += leftMax - height[l]
    else:
        r -= 1
        rightMax = max(rightMax, height[r])
        water += rightMax - height[r]
return water

```

Alternative (precompute leftMax/rightMax arrays, O(n) space):

```

def trap(height):
    n = len(height)
    left, right = [0]*n, [0]*n
    left[0] = height[0]
    for i in range(1, n): left[i] = max(left[i-1], height[i])
    right[n-1] = height[n-1]
    for i in range(n-2, -1, -1): right[i] = max(right[i+1], height[i])
    return sum(min(left[i], right[i]) - height[i] for i in range(1, n-1))

```

Dry run: at index 2 (height 0), leftMax so far is 1, rightMax is 3 → trapped = min(1, 3) - 0 = 1; sum across all such gaps → 6.

SECTION 3: Sliding Window

15. Best Time to Buy And Sell Stock

Problem: Max profit from one buy + one sell (buy before sell). **Sample I/**

O: [7, 1, 5, 3, 6, 4] → 5 (buy at 1, sell at 6)

```

def maxProfit(prices):
    minPrice = float('inf')
    profit = 0
    for p in prices:

```

```

        minPrice = min(minPrice, p)
        profit = max(profit, p - minPrice)
    return profit

```

Alternative (brute force $O(n^2)$):

```

def maxProfit(prices):
    best = 0
    for i in range(len(prices)):
        for j in range(i+1, len(prices)):
            best = max(best, prices[j]-prices[i])
    return best

```

Dry run: minPrice drops to 1 at index 1; at price 6, profit $6-1=5$, the max seen.

16. Longest Substring Without Repeating Characters

Problem: Length of longest substring with no repeated chars. **Sample I/O:** "abcabcbcb" → 3 ("abc")

```

def lengthOfLongestSubstring(s):
    seen = set()
    l = 0
    best = 0
    for r in range(len(s)):
        while s[r] in seen:
            seen.remove(s[l])
            l += 1
        seen.add(s[r])
        best = max(best, r - l + 1)
    return best

```

Alternative (hash map storing last index, skips ahead directly):

```

def lengthOfLongestSubstring(s):

```

```

last = {}
l = 0
best = 0
for r, c in enumerate(s):
    if c in last and last[c] >= l:
        l = last[c] + 1
    last[c] = r
    best = max(best, r - l + 1)
return best

```

Dry run: window grows to "abc" (len 3); hitting second 'a' shrinks window from left past first 'a', window slides but never exceeds length 3.

17. Longest Repeating Character Replacement

Problem: Longest substring of same letter after replacing up to k chars.

Sample I/O: $s = \text{"ABAB"} , k = 2 \rightarrow 4$

```

def characterReplacement(s, k):
    count = {}
    l = 0
    maxFreq = 0
    best = 0
    for r in range(len(s)):
        count[s[r]] = count.get(s[r], 0) + 1
        maxFreq = max(maxFreq, count[s[r]])
        if (r - l + 1) - maxFreq > k:
            count[s[l]] -= 1
            l += 1
        best = max(best, r - l + 1)
    return best

```

Alternative: same sliding window but recompute maxFreq via `max(count.values())` each step (simpler to write, slower $O(26n)$).

Dry run: window "ABAB", maxFreq tracks best single-letter count (2); window size 4 minus maxFreq 2 = $2 \leq k=2$, so window never shrinks $\rightarrow$

answer 4.

18. Permutation in String

Problem: Return true if `s2` contains a permutation of `s1`. **Sample I/O:**

`s1="ab", s2="eidbaooo" → True`

```
def checkInclusion(s1, s2):
    if len(s1) > len(s2): return False
    need = [0]*26
    window = [0]*26
    for c in s1: need[ord(c)-97] += 1
    for i in range(len(s2)):
        window[ord(s2[i])-97] += 1
        if i >= len(s1):
            window[ord(s2[i-len(s1)])-97] -= 1
        if window == need: return True
    return False
```

Alternative (Counter comparison each step, simpler but slower):

```
from collections import Counter
def checkInclusion(s1, s2):
    need = Counter(s1)
    for i in range(len(s2)-len(s1)+1):
        if Counter(s2[i:i+len(s1)]) == need: return True
    return False
```

Dry run: window of size 2 slides across `s2`; at position covering `"ba"`, counts match need (`a:1,b:1`) → True.

19. Minimum Window Substring

Problem: Smallest substring of `s` containing all chars of `t` (with

multiplicity). **Sample I/O:** `s="ADOBECODEBANC", t="ABC" → "BANC"`

```

def minWindow(s, t):
    if not t: return ""
    need = {}
    for c in t: need[c] = need.get(c, 0) + 1
    missing = len(t)
    l = 0
    best = (float('inf'), 0, 0)
    for r, c in enumerate(s):
        if need.get(c, 0) > 0: missing -= 1
        need[c] = need.get(c, 0) - 1
        while missing == 0:
            if r - l + 1 < best[0]: best = (r-l+1, l, r)
            need[s[l]] += 1
            if need[s[l]] > 0: missing += 1
            l += 1
    return "" if best[0] == float('inf') else s[best[1]:best[2]]

```

Alternative: two-hashmap version tracking have/need counts separately with explicit satisfied-char counter (same idea, more verbose but easier to trace). **Dry run:** window expands until all of A,B,C covered ("ADOBEC"), then shrinks from left while still valid, tracking smallest — final answer "BANC".

20. Sliding Window Maximum

Problem: Max of each size-k window as it slides across the array.

Sample I/O: nums=[1,3,-1,-3,5,3,6,7], k=3 →
[3,3,5,5,6,7]

```

from collections import deque
def maxSlidingWindow(nums, k):
    dq = deque() # stores indices, decreasing values
    res = []
    for i, n in enumerate(nums):
        while dq and nums[dq[-1]] < n: dq.pop()
        dq.append(i)
        if dq[0] <= i - k: dq.popleft()

```

```

        if i >= k - 1: res.append(nums[dq[0]])
    return res

```

Alternative (brute force, $O(n*k)$):

```

def maxSlidingWindow(nums, k):
    return [max(nums[i:i+k]) for i in range(len(nums)-k+1)]

```

Dry run: deque keeps indices of decreasing values; when window `[1, 3, -1]` forms, front of deque points at 3 → first result 3.

SECTION 4: Stack

21. Valid Parentheses

Problem: Check if brackets are balanced/correctly nested. **Sample I/O:**
`"() [] {}" → True` | `"() " → False`

```

def isValid(s):
    stack = []
    pairs = {'(': ')', '[': ']', '{': '}'
    for c in s:
        if c in pairs:
            if not stack or stack.pop() != pairs[c]: return False
        else:
            stack.append(c)
    return not stack

```

Alternative: repeatedly replace `"()"`, `"[]"`, `"{}"` with `" "` until no change, then check if empty (works but $O(n^2)$). **Dry run:** `"() "` → push `'('`, then see `']'`, pop gives `'('` but expected `'['` → mismatch → False.

22. Min Stack

Problem: Stack supporting push/pop/top/getMin all in $O(1)$. **Sample I/O:**
push -2, 0, -3; getMin() → -3; pop; getMin() → -2

```
class MinStack:
    def __init__(self):
        self.stack = []
        self.minStack = []
    def push(self, val):
        self.stack.append(val)
        m = min(val, self.minStack[-1] if self.minStack else val)
        self.minStack.append(m)
    def pop(self):
        self.stack.pop()
        self.minStack.pop()
    def top(self):
        return self.stack[-1]
    def getMin(self):
        return self.minStack[-1]
```

Alternative: single stack storing (val, currentMin) tuples instead of two parallel stacks — same idea, one structure. **Dry run:** after pushing -2, 0, -3, minStack is [-2, -2, -3]; pop removes -3 from both → minStack top is -2.

23. Evaluate Reverse Polish Notation

Problem: Evaluate an expression given in postfix (RPN) form. **Sample I/O:** ["2", "1", "+", "3", "*"] → 9

```
def evalRPN(tokens):
    stack = []
    for t in tokens:
        if t not in "+-*/":
            stack.append(int(t))
        else:
```

```

        b, a = stack.pop(), stack.pop()
        if t == '+': stack.append(a+b)
        elif t == '-': stack.append(a-b)
        elif t == '*': stack.append(a*b)
        else: stack.append(int(a/b))
    return stack[0]

```

Alternative: use `operator` module with a dict of functions (`{'+':operator.add, ...}`) instead of if/elif chain — cleaner, same logic. **Dry run:** 2, 1 pushed; '+' pops 1, 2 → pushes 3; 3 pushed; '*' pops 3, 3 → pushes 9.

24. Generate Parentheses

Problem: Generate all valid combinations of n pairs of parentheses.

Sample I/O: $n=2 \rightarrow ["()()", "()()"]$

```

def generateParenthesis(n):
    res = []
    def backtrack(cur, open_, close_):
        if len(cur) == 2*n:
            res.append(cur)
            return
        if open_ < n: backtrack(cur+'(', open_+1, close_)
        if close_ < open_: backtrack(cur+')', open_, close_+1)
    backtrack('', 0, 0)
    return res

```

Alternative: BFS with a queue of partial strings instead of recursion — same pruning rules ($open < n$, $close < open$). **Dry run:** from "", add '(' → "("; add '(' again → "(("; now close twice → "(()"; backtrack to explore "())" path too.

25. Daily Temperatures

Problem: For each day, how many days until a warmer temperature.

Sample I/O: [73,74,75,71,69,72,76,73] →
[1,1,4,2,1,1,0,0]

```
def dailyTemperatures(temps):
    res = [0]*len(temps)
    stack = [] # indices, decreasing temps
    for i, t in enumerate(temps):
        while stack and temps[stack[-1]] < t:
            j = stack.pop()
            res[j] = i - j
        stack.append(i)
    return res
```

Alternative (brute force $O(n^2)$):

```
def dailyTemperatures(temps):
    res = [0]*len(temps)
    for i in range(len(temps)):
        for j in range(i+1, len(temps)):
            if temps[j] > temps[i]:
                res[i] = j-i
                break
    return res
```

Dry run: at $i=2$ (temp 75), stack pops index 0,1 (73,74 < 75) setting $res[0]=2, res[1]=1$... continues; index 6 (76) pops everything smaller before it.

26. Car Fleet

Problem: Cars moving toward a target at different speeds/positions; count fleets that merge together. **Sample I/O:** target=12, position=[10,8,0,5,3], speed=[2,4,1,1,3] → 3

```
def carFleet(target, position, speed):
```

```

pairs = sorted(zip(position, speed), reverse=True)
stack = []
for pos, spd in pairs:
    time = (target - pos) / spd
    if not stack or time > stack[-1]:
        stack.append(time)
return len(stack)

```

Alternative: same logic but store fleets explicitly as list of times and merge with a simple loop comparing consecutive arrival times (same complexity, more verbose). **Dry run:** sorted by position descending: (10, 2), (8, 4), (5, 1), (3, 3), (0, 1); arrival times computed; a car catching up to a slower one ahead merges (doesn't add new stack entry) → 3 distinct fleets.

27. Largest Rectangle in Histogram

Problem: Largest rectangular area under a histogram. **Sample I/O:** [2, 1, 5, 6, 2, 3] → 10

```

def largestRectangleArea(heights):
    stack = [] # (index, height)
    maxArea = 0
    for i, h in enumerate(heights):
        start = i
        while stack and stack[-1][1] > h:
            idx, height = stack.pop()
            maxArea = max(maxArea, height * (i - idx))
            start = idx
        stack.append((start, h))
    for idx, h in stack:
        maxArea = max(maxArea, h * (len(heights) - idx))
    return maxArea

```

Alternative (brute force $O(n^2)$, expand around each bar):

```

def largestRectangleArea(heights):

```

```

maxArea = 0
for i in range(len(heights)):
    minH = heights[i]
    for j in range(i, len(heights)):
        minH = min(minH, heights[j])
        maxArea = max(maxArea, minH*(j-i+1))
return maxArea

```

Dry run: bars 5, 6 at indices 2,3 pop when height 2 arrives, giving area $6*1=6$ and $5*2=10$ (extended back to index 1) → max 10.

SECTION 5: Binary Search

28. Binary Search

Problem: Find target index in sorted array, or -1. **Sample I/O:**

nums=[-1,0,3,5,9,12], target=9 → 4

```

def search(nums, target):
    l, r = 0, len(nums)-1
    while l <= r:
        mid = (l+r)//2
        if nums[mid] == target: return mid
        elif nums[mid] < target: l = mid+1
        else: r = mid-1
    return -1

```

Alternative: `bisect.bisect_left` then verify the found index equals target — leans on `stdlib` instead of writing loop manually. **Dry run:** $\text{mid}=2$ (3) < 9 → search right; $\text{mid}=4$ (9) $=$ target → return 4.

29. Search a 2D Matrix

Problem: Rows sorted, first element of each row > last of previous row.
Find target. **Sample I/O:** matrix=[[1, 3, 5, 7], [10, 11, 16, 20], [23, 30, 34, 60]], target=3 → True

```
def searchMatrix(matrix, target):
    rows, cols = len(matrix), len(matrix[0])
    l, r = 0, rows*cols - 1
    while l <= r:
        mid = (l+r)//2
        val = matrix[mid//cols][mid%cols]
        if val == target: return True
        elif val < target: l = mid+1
        else: r = mid-1
    return False
```

Alternative (two-step: binary search row, then binary search within row):

```
def searchMatrix(matrix, target):
    for row in matrix:
        if row[0] <= target <= row[-1]:
            l, r = 0, len(row)-1
            while l <= r:
                mid = (l+r)//2
                if row[mid] == target: return True
                elif row[mid] < target: l = mid+1
                else: r = mid-1
    return False
```

Dry run: treat matrix as flat array of 12 elements; binary search converges on index for value 3 (row 0, col 2) → True.

30. Koko Eating Bananas

Problem: Min eating speed k so Koko finishes all piles within h hours.
Sample I/O: piles=[3, 6, 7, 11], h=8 → 4

```

import math
def minEatingSpeed(piles, h):
    l, r = 1, max(piles)
    res = r
    while l <= r:
        k = (l+r)//2
        hours = sum(math.ceil(p/k) for p in piles)
        if hours <= h:
            res = k
            r = k-1
        else:
            l = k+1
    return res

```

**Alternative (linear scan of speeds, simpler but slower $O(\max(\text{piles})n)$):*

```

import math
def minEatingSpeed(piles, h):
    for k in range(1, max(piles)+1):
        if sum(math.ceil(p/k) for p in piles) <= h:
            return k

```

Dry run: try $k=4$: $\text{hours} = 1+2+2+3=8 \leq 8 \rightarrow$ feasible, try smaller; $k=3$: $\text{hours}=1+2+3+4=10>8 \rightarrow$ infeasible $\rightarrow$ answer 4.

31. Find Minimum in Rotated Sorted Array

Problem: Find min element in a rotated sorted array (no duplicates).

Sample I/O: $[4, 5, 6, 7, 0, 1, 2] \rightarrow 0$

```

def findMin(nums):
    l, r = 0, len(nums)-1
    while l < r:
        mid = (l+r)//2
        if nums[mid] > nums[r]: l = mid+1
        else: r = mid
    return nums[l]

```

Alternative: compare `nums[mid]` to `nums[l]` instead of `nums[r]` — equivalent logic, different pivot reference. **Dry run:** `mid=3 (7) > nums[r]=2` → min is right of mid → `l=4`; eventually converge at index 4 → 0.

32. Search in Rotated Sorted Array

Problem: Search target in rotated sorted array in $O(\log n)$. **Sample I/O:**
`nums=[4,5,6,7,0,1,2], target=0` → 4

```
def search(nums, target):
    l, r = 0, len(nums)-1
    while l <= r:
        mid = (l+r)//2
        if nums[mid] == target: return mid
        if nums[l] <= nums[mid]: # left half sorted
            if nums[l] <= target < nums[mid]: r = mid-1
            else: l = mid+1
        else: # right half sorted
            if nums[mid] < target <= nums[r]: l = mid+1
            else: r = mid-1
    return -1
```

Alternative (find pivot first, then binary search the correct half):
locate rotation index via `findMin` logic, then run plain binary search offset by that pivot. **Dry run:** `mid=3 (7)`; left half `[4,5,6,7]` sorted, target 0 not in `[4,7]` → search right half → converges to index 4.

33. Time Based Key-Value Store

Problem: Store `(key, value, timestamp)`; retrieve value for key at largest timestamp $\leq$ given time. **Sample I/O:** `set("foo", "bar", 1)`;
`get("foo", 1)` → "bar"; `get("foo", 4)` → "bar"

```
class TimeMap:
```

```

def __init__(self):
    self.store = {} # key -> list of (timestamp, val)
def set(self, key, value, timestamp):
    self.store.setdefault(key, []).append((timestamp, value))
def get(self, key, timestamp):
    arr = self.store.get(key, [])
    l, r, res = 0, len(arr)-1, ""
    while l <= r:
        mid = (l+r)//2
        if arr[mid][0] <= timestamp:
            res = arr[mid][1]
            l = mid+1
        else:
            r = mid-1
    return res

```

Alternative: use `bisect.bisect_right` on a parallel list of timestamps instead of manual binary search. **Dry run:**

`arr=[(1, "bar")]`; `get(4)`: `mid=0, l<=4` → `res="bar"`, search right, loop ends → `"bar"`.

34. Median of Two Sorted Arrays

Problem: Find median of two sorted arrays in $O(\log(\min(m,n)))$. **Sample**

I/O: `nums1=[1,3], nums2=[2]` → `2.0`

```

def findMedianSortedArrays(nums1, nums2):
    if len(nums1) > len(nums2): nums1, nums2 = nums2, nums1
    m, n = len(nums1), len(nums2)
    l, r = 0, m
    while l <= r:
        i = (l+r)//2
        j = (m+n+1)//2 - i
        left1 = nums1[i-1] if i > 0 else float('-inf')
        right1 = nums1[i] if i < m else float('inf')
        left2 = nums2[j-1] if j > 0 else float('-inf')
        right2 = nums2[j] if j < n else float('inf')

```

```

    if left1 <= right2 and left2 <= right1:
        if (m+n) % 2 == 0:
            return (max(left1, left2) + min(right1, right2)) // 2
        return max(left1, left2)
    elif left1 > right2: r = i-1
    else: l = i+1

```

Alternative (simple merge, $O(m+n)$, fails the log-time requirement but easy to understand):

```

def findMedianSortedArrays(nums1, nums2):
    merged = sorted(nums1+nums2)
    n = len(merged)
    mid = n//2
    return merged[mid] if n%2 else (merged[mid-1]+merged[mid])/2

```

Dry run: merge $[1, 3] + [2]$ sorted $\rightarrow [1, 2, 3]$; odd length 3 $\rightarrow$ middle element 2 $\rightarrow 2.0$.

SECTION 6: Linked List

35. Reverse Linked List

Problem: Reverse a singly linked list. **Sample I/O:** $1 \rightarrow 2 \rightarrow 3 \rightarrow \text{None} \rightarrow 3 \rightarrow 2 \rightarrow 1 \rightarrow \text{None}$

```

def reverseList(head):
    prev = None
    while head:
        nxt = head.next
        head.next = prev
        prev = head
        head = nxt
    return prev

```

Alternative (recursive):

```
def reverseList(head):
    if not head or not head.next: return head
    newHead = reverseList(head.next)
    head.next.next = head
    head.next = None
    return newHead
```

Dry run: step 1: prev=1, head=2 (1 now points to None); step 2: prev=2->1; step 3: prev=3->2->1 → return 3->2->1.

36. Merge Two Sorted Lists

Problem: Merge two sorted linked lists into one sorted list. **Sample I/O:** 1->2->4 and 1->3->4 → 1->1->2->3->4->4

```
def mergeTwoLists(l1, l2):
    dummy = curr = ListNode()
    while l1 and l2:
        if l1.val <= l2.val:
            curr.next = l1; l1 = l1.next
        else:
            curr.next = l2; l2 = l2.next
        curr = curr.next
    curr.next = l1 or l2
    return dummy.next
```

Alternative (recursive):

```
def mergeTwoLists(l1, l2):
    if not l1: return l2
    if not l2: return l1
    if l1.val <= l2.val:
        l1.next = mergeTwoLists(l1.next, l2)
        return l1
```

```
l2.next = mergeTwoLists(l1, l2.next)
return l2
```

Dry run: compare 1, 1 → take l1's 1, advance; compare 2, 1 → take l2's 1; continues merging in order → 1, 1, 2, 3, 4, 4.

37. Reorder List

Problem: Reorder $L_0 \rightarrow L_1 \rightarrow \dots \rightarrow L_n$ to $L_0 \rightarrow L_n \rightarrow L_1 \rightarrow L_{n-1} \rightarrow \dots$ **Sample I/O:** 1→2→3→4 → 1→4→2→3

```
def reorderList(head):
    slow, fast = head, head
    while fast and fast.next:
        slow = slow.next; fast = fast.next.next
    second = slow.next
    slow.next = None
    prev = None
    while second:
        nxt = second.next
        second.next = prev
        prev = second
        second = nxt
    first, second = head, prev
    while second:
        n1, n2 = first.next, second.next
        first.next = second
        second.next = n1
        first, second = n1, n2
```

Alternative: dump all nodes into a list/array, then re-link using two pointers from front and back — simpler to write, uses $O(n)$ extra space.

Dry run: split 1, 2 | 3, 4; reverse second half → 4, 3; interleave: 1→4→2→3.

38. Remove Nth Node From End of List

Problem: Remove the nth node from the end in one pass. **Sample I/O:**

1->2->3->4->5, n=2 → 1->2->3->5

```
def removeNthFromEnd(head, n):
    dummy = ListNode(0, head)
    fast = slow = dummy
    for _ in range(n): fast = fast.next
    while fast.next:
        fast = fast.next; slow = slow.next
    slow.next = slow.next.next
    return dummy.next
```

Alternative (two-pass: count length, then remove at index len-n):

```
def removeNthFromEnd(head, n):
    length = 0
    node = head
    while node: length += 1; node = node.next
    dummy = ListNode(0, head)
    curr = dummy
    for _ in range(length - n): curr = curr.next
    curr.next = curr.next.next
    return dummy.next
```

Dry run: fast moves 2 ahead first; then both move until fast hits end;
slow lands right before node 4 → skip it → 1->2->3->5.

39. Copy List with Random Pointer

Problem: Deep copy a linked list where each node has an extra random pointer. **Sample I/O:** list with random pointers → independent deep copy with matching structure

```
def copyRandomList(head):
```

```

if not head: return None
oldToNew = {}
node = head
while node:
    oldToNew[node] = Node(node.val)
    node = node.next
node = head
while node:
    oldToNew[node].next = oldToNew.get(node.next)
    oldToNew[node].random = oldToNew.get(node.random)
    node = node.next
return oldToNew[head]

```

Alternative (interweaving copies into original list, $O(1)$ extra space):

insert each copy right after its original, wire up randoms via

`node.next.random = node.random.next`, then split the two lists apart. **Dry run:** hashmap maps each old node to its new copy; second pass wires `next/random` using the map lookups → structurally identical copy.

40. Add Two Numbers

Problem: Add two numbers represented as reversed-digit linked lists.

Sample I/O: `2->4->3 (342) + 5->6->4 (465) → 7->0->8 (807)`

```

def addTwoNumbers(l1, l2):
    dummy = curr = ListNode()
    carry = 0
    while l1 or l2 or carry:
        v1 = l1.val if l1 else 0
        v2 = l2.val if l2 else 0
        total = v1 + v2 + carry
        carry = total // 10
        curr.next = ListNode(total % 10)
        curr = curr.next
        l1 = l1.next if l1 else None
        l2 = l2.next if l2 else None

```

```
return dummy.next
```

Alternative: convert each list to an integer, add normally, convert back to a list — simple but breaks for arbitrarily huge numbers exceeding native int handling in other languages (fine in Python). **Dry run:** $2+5=7$ (carry 0); $4+6=10 \rightarrow$ digit 0, carry 1; $3+4+1=8 \rightarrow$ result digits 7, 0, 8 $\rightarrow$ 708, i.e. $342+465=807$. ✓

41. Linked List Cycle

Problem: Detect if a linked list has a cycle. **Sample I/O:** list with cycle $\rightarrow$ True

```
def hasCycle(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
        if slow == fast: return True
    return False
```

Alternative (hash set of visited nodes, $O(n)$ space):

```
def hasCycle(head):
    seen = set()
    while head:
        if head in seen: return True
        seen.add(head)
        head = head.next
    return False
```

Dry run: fast (2x speed) eventually laps slow inside the cycle, so `slow == fast` becomes true; if no cycle, fast hits None first.

42. Find the Duplicate Number

Problem: Array of $n+1$ ints in range $[1, n]$, one duplicate; find it without modifying array, $O(1)$ space. **Sample I/O:** `[1, 3, 4, 2, 2]` → 2

```
def findDuplicate(nums):
    slow = fast = 0
    while True:
        slow = nums[slow]
        fast = nums[nums[fast]]
        if slow == fast: break
    slow2 = 0
    while slow2 != slow:
        slow2 = nums[slow2]
        slow = nums[slow]
    return slow
```

Alternative (sort or hash set, simpler but violates $O(1)$ -space/no-modify constraint):

```
def findDuplicate(nums):
    seen = set()
    for n in nums:
        if n in seen: return n
        seen.add(n)
```

Dry run: treat array as a linked list via indices; Floyd's cycle detection finds meeting point, then second phase finds cycle entrance = duplicate value 2.

43. LRU Cache

Problem: Design a cache with $O(1)$ get/put, evicting least-recently-used item when full. **Sample I/O:** `capacity=2; put(1,1), put(2,2), get(1)→1, put(3,3) evicts key 2, get(2)→-1`

```

class LRUCache:
    def __init__(self, capacity):
        self.cap = capacity
        self.cache = {} # OrderedDict-like via dict (Pyt
        from collections import OrderedDict
        self.cache = OrderedDict()
    def get(self, key):
        if key not in self.cache: return -1
        self.cache.move_to_end(key)
        return self.cache[key]
    def put(self, key, value):
        if key in self.cache: self.cache.move_to_end(key)
        self.cache[key] = value
        if len(self.cache) > self.cap:
            self.cache.popitem(last=False)

```

Alternative (manual doubly linked list + hashmap, no library, true $O(1)$ guaranteed): maintain head/tail sentinel nodes; hashmap maps key→node; on access, unlink node and re-insert at front; on overflow, remove node before tail. **Dry run:** after put(1,1),put(2,2): order [1, 2]; get(1) moves 1 to end → [2, 1]; put(3,3) evicts front (2) → cache has {1, 3} → get(2) → -1.

44. Merge K Sorted Lists

Problem: Merge k sorted linked lists into one sorted list. **Sample I/O:**

[[1, 4, 5], [1, 3, 4], [2, 6]] → [1, 1, 2, 3, 4, 4, 5, 6]

```

import heapq
def mergeKLists(lists):
    heap = []
    for i, node in enumerate(lists):
        if node: heapq.heappush(heap, (node.val, i, node))
    dummy = curr = ListNode()
    while heap:
        val, i, node = heapq.heappop(heap)
        curr.next = node

```

```

        curr = curr.next
        if node.next: heapq.heappush(heap, (node.next.val, node.next))
    return dummy.next

```

Alternative (divide and conquer, merge pairs of lists using mergeTwoLists):

```

def mergeKLists(lists):
    if not lists: return None
    while len(lists) > 1:
        merged = []
        for i in range(0, len(lists), 2):
            l1 = lists[i]
            l2 = lists[i+1] if i+1 < len(lists) else None
            merged.append(mergeTwoLists(l1, l2))
        lists = merged
    return lists[0]

```

Dry run: heap always pops the globally smallest head node next, pushing its successor back in → produces fully sorted merge
 1, 1, 2, 3, 4, 4, 5, 6.

45. Reverse Nodes in K Group

Problem: Reverse nodes of a linked list k at a time. **Sample I/O:** 1→2→3→4→5, k=2 → 2→1→4→3→5

```

def reverseKGroup(head, k):
    node = head
    for _ in range(k):
        if not node: return head
        node = node.next
    prev = reverseKGroup(node, k)
    curr = head
    for _ in range(k):
        nxt = curr.next
        curr.next = prev
        prev = curr
        curr = nxt
    return prev

```

```
prev = curr
curr = nxt
return prev
```

Alternative (iterative with dummy node, tracking group boundaries): repeatedly find the k-th node ahead, reverse that segment in place using a mini reversal loop, and relink to the previous group's tail.
Dry run: check k=2 nodes exist (1, 2); recursively reverse rest starting at 3; reverse first pair 1, 2 → 2 → 1, attach to recursively-reversed remainder → 2 → 1 → 4 → 3 → 5.

SECTION 7: Trees

46. Invert Binary Tree

Problem: Swap every left/right child in a binary tree. **Sample I/O:**

[4, 2, 7, 1, 3, 6, 9] → [4, 7, 2, 9, 6, 3, 1]

```
def invertTree(root):
    if not root: return None
    root.left, root.right = invertTree(root.right), invertTree(root.left)
    return root
```

Alternative (iterative BFS with queue):

```
from collections import deque
def invertTree(root):
    if not root: return None
    q = deque([root])
    while q:
        node = q.popleft()
        node.left, node.right = node.right, node.left
        if node.left: q.append(node.left)
        if node.right: q.append(node.right)
```

```
return root
```

Dry run: root 4's children swap to (7, 2); recursion swaps 7's children (9, 6) and 2's children (3, 1) → mirrored tree.

47. Maximum Depth of Binary Tree

Problem: Return the max depth (number of nodes on longest root-to-leaf path). **Sample I/O:** [3, 9, 20, null, null, 15, 7] → 3

```
def maxDepth(root):  
    if not root: return 0  
    return 1 + max(maxDepth(root.left), maxDepth(root.right))
```

Alternative (iterative BFS level counting):

```
from collections import deque  
def maxDepth(root):  
    if not root: return 0  
    q = deque([root])  
    depth = 0  
    while q:  
        depth += 1  
        for _ in range(len(q)):  
            node = q.popleft()  
            if node.left: q.append(node.left)  
            if node.right: q.append(node.right)  
    return depth
```

Dry run: depth(3) = 1+max(depth(9), depth(20));
depth(20)=1+max(depth(15),depth(7))=2 → total 1+2=3.

48. Diameter of Binary Tree

Problem: Longest path between any two nodes (in edges), may not pass through root. **Sample I/O:** [1, 2, 3, 4, 5] → 3

```
def diameterOfBinaryTree(root):
    diameter = 0
    def depth(node):
        nonlocal diameter
        if not node: return 0
        l, r = depth(node.left), depth(node.right)
        diameter = max(diameter, l+r)
        return 1 + max(l, r)
    depth(root)
    return diameter
```

Alternative: compute depth recursively without early combination, storing depths in a memo dict, then separately scan all nodes computing `depth(left)+depth(right)` — same result, more calls ($O(n^2)$ worst case). **Dry run:** at node 1, left depth=2 (path through 4 or 5), right depth=1 → diameter candidate 3, which is the max found.

49. Balanced Binary Tree

Problem: Determine if a binary tree is height-balanced (subtree heights differ ≤ 1 everywhere). **Sample I/O:** [3, 9, 20, null, null, 15, 7] → True

```
def isBalanced(root):
    def height(node):
        if not node: return 0
        l = height(node.left)
        if l == -1: return -1
        r = height(node.right)
        if r == -1: return -1
        if abs(l-r) > 1: return -1
        return 1 + max(l, r)
    return height(root) != -1
```

Alternative (naive, recompute height separately at every node — $O(n^2)$):

```
def isBalanced(root):
    def height(node):
        if not node: return 0
        return 1 + max(height(node.left), height(node.right))
    if not root: return True
    if abs(height(root.left) - height(root.right)) > 1: return False
    return isBalanced(root.left) and isBalanced(root.right)
```

Dry run: every subtree height difference ≤ 1 , so `-1` sentinel never triggers, final `height(root)=3 != -1 → True`.

50. Same Tree

Problem: Check if two binary trees are structurally identical with same values. **Sample I/O:** `p=[1,2,3], q=[1,2,3] → True`

```
def isSameTree(p, q):
    if not p and not q: return True
    if not p or not q or p.val != q.val: return False
    return isSameTree(p.left, q.left) and isSameTree(p.right, q.right)
```

Alternative (serialize both trees to strings including null markers, compare strings):

```
def isSameTree(p, q):
    def serialize(node):
        if not node: return "null"
        return f"{node.val},{serialize(node.left)},{serialize(node.right)}"
    return serialize(p) == serialize(q)
```

Dry run: roots match (`1==1`), recurse left (`2==2`, both leaves) and right (`3==3`, both leaves) → True.

51. Subtree of Another Tree

Problem: Check if `subRoot` is a subtree of `root`. **Sample I/O:**

`root=[3,4,5,1,2]`, `subRoot=[4,1,2]` → `True`

```
def isSubtree(root, subRoot):
    def sameTree(a, b):
        if not a and not b: return True
        if not a or not b or a.val != b.val: return False
        return sameTree(a.left, b.left) and sameTree(a.right, b.right)
    if not root: return False
    if sameTree(root, subRoot): return True
    return isSubtree(root.left, subRoot) or isSubtree(root.right, subRoot)
```

Alternative: serialize both trees to strings (with unique delimiters/null markers) and check if `subRoot`'s string is a substring of `root`'s string.

Dry run: at `root`'s left child 4, `sameTree(4-subtree, subRoot)` matches fully → return `True` without checking further nodes.

52. Lowest Common Ancestor of a BST

Problem: Find LCA of two nodes in a binary search tree. **Sample I/O:**

`root=[6,2,8,0,4,7,9]`, `p=2`, `q=8` → `6`

```
def lowestCommonAncestor(root, p, q):
    curr = root
    while curr:
        if p.val < curr.val and q.val < curr.val: curr = curr.left
        elif p.val > curr.val and q.val > curr.val: curr = curr.right
        else: return curr
```

Alternative (recursive, same BST-property logic):

```
def lowestCommonAncestor(root, p, q):
    if p.val < root.val and q.val < root.val: return lowestCommonAncestor(root.left, p, q)
    if p.val > root.val and q.val > root.val: return lowestCommonAncestor(root.right, p, q)
    return root
```

```
if p.val > root.val and q.val > root.val: return lower
return root
```

Dry run: at root 6: $p=2 < 6$ but $q=8 > 6 \rightarrow$ split point $\rightarrow$ return 6 immediately.

53. Binary Tree Level Order Traversal

Problem: Return node values grouped by level (BFS). **Sample I/O:**

`[3,9,20,null,null,15,7]` $\rightarrow$ `[[3],[9,20],[15,7]]`

```
from collections import deque
def levelOrder(root):
    if not root: return []
    res, q = [], deque([root])
    while q:
        level = []
        for _ in range(len(q)):
            node = q.popleft()
            level.append(node.val)
            if node.left: q.append(node.left)
            if node.right: q.append(node.right)
        res.append(level)
    return res
```

Alternative (DFS tracking depth, append to res[depth]):

```
def levelOrder(root):
    res = []
    def dfs(node, depth):
        if not node: return
        if depth == len(res): res.append([])
        res[depth].append(node.val)
        dfs(node.left, depth+1)
        dfs(node.right, depth+1)
    dfs(root, 0)
    return res
```

Dry run: queue processes [3] → level [3]; then [9, 20] → level [9, 20]; then [15, 7] → level [15, 7].

54. Binary Tree Right Side View

Problem: Return values visible from the right side, top to bottom.

Sample I/O: [1, 2, 3, null, 5, null, 4] → [1, 3, 4]

```
from collections import deque
def rightSideView(root):
    if not root: return []
    res, q = [], deque([root])
    while q:
        size = len(q)
        for i in range(size):
            node = q.popleft()
            if i == size-1: res.append(node.val)
            if node.left: q.append(node.left)
            if node.right: q.append(node.right)
    return res
```

Alternative (DFS, visit right before left, take first node seen at each depth):

```
def rightSideView(root):
    res = []
    def dfs(node, depth):
        if not node: return
        if depth == len(res): res.append(node.val)
        dfs(node.right, depth+1)
        dfs(node.left, depth+1)
    dfs(root, 0)
    return res
```

Dry run: level 0: [1] → last is 1; level 1: [2, 3] → last is 3; level 2: [5, 4] → last is 4 → [1, 3, 4].

55. Count Good Nodes in Binary Tree

Problem: Count nodes where value $\geq$ all ancestors' values (node is "good"). **Sample I/O:** [3, 1, 4, 3, null, 1, 5] $\rightarrow$ 4

```
def goodNodes(root):
    def dfs(node, maxSoFar):
        if not node: return 0
        count = 1 if node.val >= maxSoFar else 0
        maxSoFar = max(maxSoFar, node.val)
        return count + dfs(node.left, maxSoFar) + dfs(node.right, maxSoFar)
    return dfs(root, root.val)
```

Alternative (BFS carrying maxSoFar in the queue alongside each node):

```
from collections import deque
def goodNodes(root):
    count = 0
    q = deque([(root, root.val)])
    while q:
        node, maxSoFar = q.popleft()
        if node.val >= maxSoFar: count += 1
        maxSoFar = max(maxSoFar, node.val)
        if node.left: q.append((node.left, maxSoFar))
        if node.right: q.append((node.right, maxSoFar))
    return count
```

Dry run: path 3 $\rightarrow$ 1 $\rightarrow$ 4 $\rightarrow$ 5 and 3 $\rightarrow$ 3 etc; good nodes are 3, 4, 3 (left child), 5 $\rightarrow$ count 4.

56. Validate Binary Search Tree

Problem: Check if a tree satisfies the BST property. **Sample I/O:** [2, 1, 3] $\rightarrow$ True | [5, 1, 4, null, null, 3, 6] $\rightarrow$ False

```
def isValidBST(root):
    def valid(node, low, high):
        if not node: return True
        if not (low < node.val < high): return False
        return valid(node.left, low, node.val) and valid(
            node.right, node.val, high)
    return valid(root, float('-inf'), float('inf'))
```

Alternative (inorder traversal must be strictly increasing):

```
def isValidBST(root):
    prev = float('-inf')
    def inorder(node):
        nonlocal prev
        if not node: return True
        if not inorder(node.left): return False
        if node.val <= prev: return False
        prev = node.val
        return inorder(node.right)
    return inorder(root)
```

Dry run: node 4's right subtree has 3, but 3 must be >4 (bounds (4, inf)) $\rightarrow$ fails $\rightarrow$ False.

57. Kth Smallest Element in a BST

Problem: Return kth smallest value (1-indexed). **Sample I/O:**

root=[3,1,4,null,2], k=1 $\rightarrow$ 1

```
def kthSmallest(root, k):
    stack = []
    curr = root
    while True:
        while curr:
            stack.append(curr)
            curr = curr.left
        curr = stack.pop()
        k -= 1
```

```
if k == 0: return curr.val
curr = curr.right
```

Alternative (full inorder traversal into a list, index in):

```
def kthSmallest(root, k):
    res = []
    def inorder(node):
        if not node: return
        inorder(node.left)
        res.append(node.val)
        inorder(node.right)
    inorder(root)
    return res[k-1]
```

Dry run: inorder visits 1, 2, 3, 4; k=1 → first value visited is 1 → return 1.

58. Construct Binary Tree from Preorder and Inorder Traversal

Problem: Rebuild the tree given its preorder and inorder sequences.

Sample I/O: preorder=[3, 9, 20, 15, 7],

inorder=[9, 3, 15, 20, 7] → [3, 9, 20, null, null, 15, 7]

```
def buildTree(preorder, inorder):
    if not preorder: return None
    rootVal = preorder[0]
    root = TreeNode(rootVal)
    mid = inorder.index(rootVal)
    root.left = buildTree(preorder[1:mid+1], inorder[:mid])
    root.right = buildTree(preorder[mid+1:], inorder[mid+1:])
    return root
```

Alternative (index map + shared pointer for O(n) instead of slicing, avoids O(n²) worst case): precompute {val: idx} for inorder, pass

index ranges instead of slicing arrays, use a preorder pointer that advances globally. **Dry run:** preorder[0]=3 is root; inorder splits into [9] (left) and [15, 20, 7] (right); recursively build 9 as left leaf, and [20, 15, 7]/[15, 20, 7] on the right into 20 with children 15, 7.

59. Binary Tree Maximum Path Sum

Problem: Max sum of any path between two nodes (path need not pass through root). **Sample I/O:** [-10, 9, 20, null, null, 15, 7] → 42

```
def maxPathSum(root):
    best = float('-inf')
    def dfs(node):
        nonlocal best
        if not node: return 0
        l = max(dfs(node.left), 0)
        r = max(dfs(node.right), 0)
        best = max(best, node.val + l + r)
        return node.val + max(l, r)
    dfs(root)
    return best
```

Alternative: same recursive idea but return (bestPathThroughSubtree, bestDownwardPath) as a tuple explicitly instead of a nonlocal variable — clearer separation, same result. **Dry run:** at node 20: left=15, right=7, best candidate 20+15+7=42; upward return is 20+15=35; root -10 isn't worth including → global best stays 42.

60. Serialize and Deserialize Binary Tree

Problem: Convert a tree to a string and back, preserving structure.

Sample I/O: [1, 2, 3, null, null, 4, 5] → serialized string → same tree on deserialize

```

def serialize(root):
    res = []
    def dfs(node):
        if not node:
            res.append("N")
            return
        res.append(str(node.val))
        dfs(node.left)
        dfs(node.right)
    dfs(root)
    return ','.join(res)

def deserialize(data):
    vals = iter(data.split(','))
    def dfs():
        v = next(vals)
        if v == "N": return None
        node = TreeNode(int(v))
        node.left = dfs()
        node.right = dfs()
        return node
    return dfs()

```

Alternative: use level-order (BFS) format like LeetCode's display format, with a queue on both serialize and deserialize — more verbose, matches common tree-string convention. **Dry run:** preorder serialize of `[1,2,3,null,null,4,5]` → `"1,2,N,N,3,4,N,N,5,N,N"`; deserialize consumes tokens in the same order to rebuild identical structure.

SECTION 8: Tries

61. Implement Trie (Prefix Tree)

Problem: Support insert, search (exact word), and startsWith (prefix) in

O(L). **Sample I/O:** insert("apple"); search("apple")→True;
search("app")→False; startsWith("app")→True

```
class TrieNode:
    def __init__(self):
        self.children = {}
        self.isEnd = False

class Trie:
    def __init__(self):
        self.root = TrieNode()
    def insert(self, word):
        node = self.root
        for c in word:
            node = node.children.setdefault(c, TrieNode())
        node.isEnd = True
    def search(self, word):
        node = self.root
        for c in word:
            if c not in node.children: return False
            node = node.children[c]
        return node.isEnd
    def startsWith(self, prefix):
        node = self.root
        for c in prefix:
            if c not in node.children: return False
            node = node.children[c]
        return True
```

Alternative: store words in a plain set/list and use Python's `str.startswith` for prefix checks — simple but $O(n \cdot L)$ per query instead of $O(L)$. **Dry run:** insert "apple" builds chain $a \rightarrow p \rightarrow p \rightarrow l \rightarrow e$ marking `e` as end; search("app") reaches node `p` but `isEnd=False` → False; startsWith("app") just needs the path to exist → True.

62. Design Add and Search Words Data Structure

Problem: Trie supporting wildcard `.` in search (matches any single letter). **Sample I/O:** `addWord("bad"); search("b.d") → True`

```
class WordDictionary:
    def __init__(self):
        self.root = {}
    def addWord(self, word):
        node = self.root
        for c in word:
            node = node.setdefault(c, {})
        node['$'] = True
    def search(self, word):
        def dfs(node, i):
            if i == len(word): return '$' in node
            c = word[i]
            if c == '.':
                return any(dfs(child, i+1) for k, child in node.items())
            if c not in node: return False
            return dfs(node[c], i+1)
        return dfs(self.root, 0)
```

Alternative: store all words in a list grouped by length; for search with `.`, filter same-length words and check char-by-char match — avoids trie but $O(n \cdot L)$ per query. **Dry run:** `"b.d"` walks `b` → then `.` tries all children of that node (`a`) → then `d` matches end marker `$` → `True`.

63. Word Search II

Problem: Given a board and list of words, find all words present as paths of adjacent cells. **Sample I/O:** board with letters, `words=["oath","pea","eat","rain"] → ["eat","oath"]`

```
def findWords(board, words):
    root = {}
    for w in words:
        node = root
        for c in w: node = node.setdefault(c, {})
```

```

node['$'] = w

rows, cols = len(board), len(board[0])
res = set()
def dfs(r, c, node):
    char = board[r][c]
    if char not in node: return
    nxt = node[char]
    if '$' in nxt: res.add(nxt['$'])
    board[r][c] = '#'
    for dr, dc in [(0,1), (0,-1), (1,0), (-1,0)]:
        nr, nc = r+dr, c+dc
        if 0<=nr<rows and 0<=nc<cols:
            dfs(nr, nc, nxt)
    board[r][c] = char
for r in range(rows):
    for c in range(cols):
        dfs(r, c, root)
return list(res)

```

Alternative (brute force, run separate word-search DFS per word instead of a shared trie): $O(\text{words} \times \text{rows} \times \text{cols} \times 4^L)$ — much slower when word list is large since it re-explores the board per word. **Dry run:** trie holds all words; DFS from each board cell simultaneously matches all words sharing prefixes, marking cell '#' to avoid reuse, restoring after backtrack.

SECTION 9: Heap / Priority Queue

64. Kth Largest Element in a Stream

Problem: Design a class that returns the kth largest element after each add. **Sample I/O:** $k=3$, `nums=[4, 5, 8, 2]`; `add(3)→4`

```
import heapq
```

```

class KthLargest:
    def __init__(self, k, nums):
        self.k = k
        self.heap = nums
        heapq.heapify(self.heap)
        while len(self.heap) > k: heapq.heappop(self.heap)
    def add(self, val):
        heapq.heappush(self.heap, val)
        if len(self.heap) > self.k: heapq.heappop(self.heap)
        return self.heap[0]

```

Alternative: keep a sorted list and use `bisect.insort` to insert, then read `list[-k]` — $O(n)$ insert but simple to reason about. **Dry run:** heap keeps only top-3 values; after `add(3)`, heap holds `{3, 4, 5, 8}` trimmed to size 3 `{4, 5, 8}` → smallest (kth largest) is 4.

65. Last Stone Weight

Problem: Repeatedly smash two heaviest stones; return final weight (or 0). **Sample I/O:** `[2, 7, 4, 1, 8, 1]` → 1

```

import heapq
def lastStoneWeight(stones):
    heap = [-s for s in stones]
    heapq.heapify(heap)
    while len(heap) > 1:
        a, b = -heapq.heappop(heap), -heapq.heappop(heap)
        if a != b: heapq.heappush(heap, -(a-b))
    return -heap[0] if heap else 0

```

Alternative (sort each time, $O(n^2 \log n)$):

```

def lastStoneWeight(stones):
    stones = sorted(stones)
    while len(stones) > 1:
        a, b = stones.pop(), stones.pop()
        if a != b: stones.append(a-b)

```

```
stones.sort()
return stones[0] if stones else 0
```

Dry run: smash 8, 7→1; smash 4, 2→2; smash 2, 1→1; smash 1, 1→0 → final weight 1.

66. K Closest Points to Origin

Problem: Return k points closest to origin (0,0). **Sample I/O:**

points=[[1,3], [-2,2]], k=1 → [[-2,2]]

```
import heapq
def kClosest(points, k):
    heap = [(x*x+y*y, x, y) for x, y in points]
    heapq.heapify(heap)
    return [[x,y] for _, x, y in heapq.nsmallest(k, heap)]
```

Alternative (sort all points by distance, take first k):

```
def kClosest(points, k):
    points.sort(key=lambda p: p[0]**2 + p[1]**2)
    return points[:k]
```

Dry run: distances: (1,3)→10, (-2,2)→8; smallest is 8 → return [[-2,2]].

67. Kth Largest Element in an Array

Problem: Find the kth largest element (1st largest = max) in an unsorted array. **Sample I/O:** [3,2,1,5,6,4], k=2 → 5

```
import heapq
def findKthLargest(nums, k):
    return heapq.nlargest(k, nums)[-1]
```

Alternative (Quickselect, average $O(n)$):

```
import random
def findKthLargest(nums, k):
    target = len(nums) - k
    def quickselect(l, r):
        pivot = nums[r]
        p = l
        for i in range(l, r):
            if nums[i] <= pivot:
                nums[i], nums[p] = nums[p], nums[i]
                p += 1
        nums[p], nums[r] = nums[r], nums[p]
        if p == target: return nums[p]
        elif p < target: return quickselect(p+1, r)
        else: return quickselect(l, p-1)
    return quickselect(0, len(nums)-1)
```

Dry run: sorted descending [6, 5, 4, 3, 2, 1]; 2nd largest is 5.

68. Task Scheduler

Problem: Min time to run all tasks with cooldown n between same-type tasks (idle allowed). **Sample I/O:**

tasks=["A", "A", "A", "B", "B", "B"], n=2 → 8

```
from collections import Counter
import heapq
def leastInterval(tasks, n):
    count = Counter(tasks)
    maxHeap = [-c for c in count.values()]
    heapq.heapify(maxHeap)
    time = 0
    q = [] # (available_time, count)
    while maxHeap or q:
        time += 1
        if maxHeap:
```

```

        c = 1 + heapq.heappop(maxHeap)
        if c: q.append((time+n, c))
    if q and q[0][0] == time:
        heapq.heappush(maxHeap, q.pop(0)[1])
    return time

```

Alternative (math formula, no simulation):

```

from collections import Counter
def leastInterval(tasks, n):
    count = Counter(tasks)
    maxCount = max(count.values())
    numMax = sum(1 for c in count.values() if c == maxCount)
    return max(len(tasks), (maxCount-1)*(n+1) + numMax)

```

Dry run (formula): maxCount=3 (A or B), numMax=2 →
 $(3-1) * (2+1) + 2 = 6+2 = 8.$

69. Design Twitter

Problem: Design simplified Twitter: postTweet, getNewsFeed (10 most recent from followees+self), follow, unfollow. **Sample I/O:**
 postTweet(1,5); getNewsFeed(1) → [5]

```

import heapq
from collections import defaultdict
class Twitter:
    def __init__(self):
        self.time = 0
        self.tweets = defaultdict(list) # user -> [(time, tweetId)]
        self.following = defaultdict(set)
    def postTweet(self, userId, tweetId):
        self.tweets[userId].append((self.time, tweetId))
        self.time -= 1 # so heap pops most recent first
    def getNewsFeed(self, userId):
        heap = []
        users = self.following[userId] | {userId}

```

```

    for u in users:
        if self.tweets[u]:
            idx = len(self.tweets[u]) - 1
            t, tid = self.tweets[u][idx]
            heap.append((t, tid, u, idx-1))
    heapq.heapify(heap)
    res = []
    while heap and len(res) < 10:
        t, tid, u, idx = heapq.heappop(heap)
        res.append(tid)
        if idx >= 0:
            nt, ntid = self.tweets[u][idx]
            heapq.heappush(heap, (nt, ntid, u, idx-1))
    return res
def follow(self, followerId, followeeId):
    if followerId != followeeId: self.following[followerId].add(followeeId)
def unfollow(self, followerId, followeeId):
    self.following[followerId].discard(followeeId)

```

Alternative: collect all tweets from followed users + self into one list, sort by timestamp descending, slice top 10 — simpler, but $O(n \log n)$ per feed call vs heap merge. **Dry run:** postTweet gives tweet 5 timestamp 0 (then internal counter goes negative for ordering); getNewsFeed for user with only own tweets returns [5].

70. Find Median from Data Stream

Problem: Support adding numbers and querying the median at any point, in $O(\log n)$ add. **Sample I/O:** addNum(1), addNum(2) → median 1.5; addNum(3) → median 2

```

import heapq
class MedianFinder:
    def __init__(self):
        self.small = [] # max-heap (negated)
        self.large = [] # min-heap
    def addNum(self, num):

```

```

        heapq.heappush(self.small, -num)
        heapq.heappush(self.large, -heapq.heappop(self.small))
        if len(self.large) > len(self.small):
            heapq.heappush(self.small, -heapq.heappop(self.large))
    def findMedian(self):
        if len(self.small) > len(self.large): return -self.small[0]
        return (-self.small[0] + self.large[0]) / 2

```

Alternative (insert into a sorted list via `bisect.insort`, $O(n)$ per add but $O(1)$ median lookup):

```

import bisect
class MedianFinder:
    def __init__(self):
        self.data = []
    def addNum(self, num):
        bisect.insort(self.data, num)
    def findMedian(self):
        n = len(self.data)
        mid = n//2
        return self.data[mid] if n%2 else (self.data[mid-1] + self.data[mid]) / 2

```

Dry run: after adding 1,2: `small={-1}` (holds 1), `large={2}` → balanced, median=(1+2)/2=1.5; add 3 rebalances so `small` has 2 elements → median = top of small = 2.

SECTION 10: Backtracking

71. Subsets

Problem: Return all possible subsets (the power set), no duplicates in input. **Sample I/O:** `[1, 2, 3]` → `[[], [1], [2], [1, 2], [3], [1, 3], [2, 3], [1, 2, 3]]` (order may vary)

```
def subsets(nums):
    res = []
    def backtrack(i, cur):
        if i == len(nums):
            res.append(cur[:])
            return
        cur.append(nums[i])
        backtrack(i+1, cur)
        cur.pop()
        backtrack(i+1, cur)
    backtrack(0, [])
    return res
```

Alternative (iterative, build up by doubling):

```
def subsets(nums):
    res = [[]]
    for n in nums:
        res += [subset + [n] for subset in res]
    return res
```

Dry run: at each index, branch into "include" and "exclude"; for [1,2,3] this produces all 8 combinations of include/exclude decisions.

72. Combination Sum

Problem: Find all unique combinations summing to target; numbers may repeat. **Sample I/O:** candidates=[2,3,6,7], target=7 → [[2,2,3], [7]]

```
def combinationSum(candidates, target):
    res = []
    def backtrack(i, cur, total):
        if total == target:
            res.append(cur[:]); return
        if i >= len(candidates) or total > target: return
        cur.append(candidates[i])
```

```

        backtrack(i, cur, total+candidates[i]) # reuse s
        cur.pop()
        backtrack(i+1, cur, total)
    backtrack(0, [], 0)
    return res

```

Alternative: sort candidates first, then prune early when `total + candidates[i] > target` (branch cutting for efficiency) — same core recursion, faster on large inputs. **Dry run:** picking 2, 2, 3 sums to 7 → recorded; picking 7 alone also sums to 7 → recorded; other branches exceed or fall short of target and are discarded.

73. Permutations

Problem: Return all permutations of distinct integers. **Sample I/O:**

`[1, 2, 3] → [[1, 2, 3], [1, 3, 2], [2, 1, 3], [2, 3, 1], [3, 1, 2], [3, 2, 1]]`

```

def permute(nums):
    res = []
    def backtrack(cur, remaining):
        if not remaining:
            res.append(cur[:]); return
        for i in range(len(remaining)):
            cur.append(remaining[i])
            backtrack(cur, remaining[:i]+remaining[i+1:])
            cur.pop()
    backtrack([], nums)
    return res

```

Alternative: from `itertools import permutations`; return `[list(p) for p in permutations(nums)]` — leans on `stdlib`.

Dry run: choosing 1 first, then recursively permute `[2, 3] → [1, 2, 3]` and `[1, 3, 2]`; repeat starting with 2, then 3, covering all 6 orderings.

74. Subsets II

Problem: Power set of an array that may contain duplicates; no duplicate subsets in output. **Sample I/O:** `[1, 2, 2] → [[], [1], [1, 2], [1, 2, 2], [2], [2, 2]]`

```
def subsetsWithDup(nums):
    nums.sort()
    res = []
    def backtrack(i, cur):
        res.append(cur[:])
        for j in range(i, len(nums)):
            if j > i and nums[j] == nums[j-1]: continue
            cur.append(nums[j])
            backtrack(j+1, cur)
            cur.pop()
    backtrack(0, [])
    return res
```

Alternative: generate all subsets ignoring duplicates (as in plain Subsets), then dedupe using a set of sorted tuples — simpler logic, wasted work on duplicates. **Dry run:** sorted `[1, 2, 2]`; after choosing first 2 at index 1, skip index 2's duplicate 2 at the *same recursion level* to avoid a repeated `[1, 2]` — but nested recursion still reaches `[1, 2, 2]` correctly.

75. Combination Sum II

Problem: Combinations summing to target using each number once; input may have duplicates. **Sample I/O:** `candidates=[10,1,2,7,6,1,5], target=8 → [[1,1,6], [1,2,5], [1,7], [2,6]]`

```
def combinationSum2(candidates, target):
    candidates.sort()
    res = []
    def backtrack(i, cur, total):
```

```

        if total == target:
            res.append(cur[:]); return
    for j in range(i, len(candidates)):
        if j > i and candidates[j] == candidates[j-1]:
            if total + candidates[j] > target: break
        cur.append(candidates[j])
        backtrack(j+1, cur, total+candidates[j])
        cur.pop()
    backtrack(0, [], 0)
    return res

```

Alternative: use a `Counter` of values instead of sorting+skipping duplicates; iterate distinct values and try `0..count` copies of each — avoids index-based duplicate skip logic. **Dry run:** sorted `[1,1,2,5,6,7,10]`; picking both `1s + 6 = 8` → recorded; the duplicate-skip `j>i and candidates[j]==candidates[j-1]` prevents re-picking the second `1` as a *new* starting choice at the same level (but still allows using both when chosen consecutively via recursion into `j+1`).

76. Word Search

Problem: Check if a word can be constructed from adjacent cells (no cell reused) in a grid. **Sample I/O:** `board=[["A","B","C","E"],["S","F","C","S"],["A","D","E","E"]]`, `word="ABCCED"` → `True`

```

def exist(board, word):
    rows, cols = len(board), len(board[0])
    def dfs(r, c, i):
        if i == len(word): return True
        if r<0 or c<0 or r>=rows or c>=cols or board[r][c] != word[i]:
            return False
        board[r][c] = '#'
        found = (dfs(r+1,c,i+1) or dfs(r-1,c,i+1) or dfs(r,c+1,i+1) or dfs(r,c-1,i+1))
        board[r][c] = word[i]
        return found
    for r in range(rows):
        for c in range(cols):
            if dfs(r, c, 0): return True
    return False

```

```

for r in range(rows):
    for c in range(cols):
        if dfs(r, c, 0): return True
return False

```

Alternative: track visited cells with a separate `set()` of `(r, c)` pairs instead of mutating the board in place — avoids modifying input, slightly more memory. **Dry run:** DFS starting at `A(0, 0)` follows path `A→B→C→C→E→D` matching each letter of `"ABCCED"`, marking cells visited then restoring → path found → `True`.

77. Palindrome Partitioning

Problem: Partition a string so every substring is a palindrome; return all partitions. **Sample I/O:** `"aab" → [ ["a", "a", "b"], ["aa", "b"] ]`

```

def partition(s):
    res = []
    def isPalin(sub): return sub == sub[::-1]
    def backtrack(i, cur):
        if i == len(s):
            res.append(cur[:]); return
        for j in range(i, len(s)):
            if isPalin(s[i:j+1]):
                cur.append(s[i:j+1])
                backtrack(j+1, cur)
                cur.pop()
    backtrack(0, [])
    return res

```

Alternative (precompute palindrome table via DP to avoid re-checking substrings):

```

def partition(s):
    n = len(s)
    dp = [[False]*n for _ in range(n)]
    for i in range(n): dp[i][i] = True

```

```

for length in range(2, n+1):
    for i in range(n-length+1):
        j = i+length-1
        dp[i][j] = s[i]==s[j] and (length==2 or dp[i+1][j-1])
res = []
def backtrack(i, cur):
    if i == n: res.append(cur[:]); return
    for j in range(i, n):
        if dp[i][j]:
            cur.append(s[i:j+1]); backtrack(j+1, cur)
backtrack(0, [])
return res

```

Dry run: "aab": try "a" (palindrome) then partition "ab" → gives "a", "b"; try "aa" (palindrome) then partition "b" → gives "aa", "b" → two total results.

78. Letter Combinations of a Phone Number

Problem: Return all letter combinations a digit string could represent (phone keypad). **Sample I/O:** "23" →

["ad", "ae", "af", "bd", "be", "bf", "cd", "ce", "cf"]

```

def letterCombinations(digits):
    if not digits: return []
    mapping = {"2": "abc", "3": "def", "4": "ghi", "5": "jkl", "6": "mno", "7": "pqrs", "8": "tuv", "9": "wxyz"}
    res = []
    def backtrack(i, cur):
        if i == len(digits):
            res.append(''.join(cur)); return
        for c in mapping[digits[i]]:
            cur.append(c)
            backtrack(i+1, cur)
            cur.pop()
    backtrack(0, [])
    return res

```

Alternative (iterative, build up via Cartesian product):

```
from itertools import product
def letterCombinations(digits):
    if not digits: return []
    mapping = {"2": "abc", "3": "def", "4": "ghi", "5": "jkl", "6": "lmn", "7": "opqr", "8": "stuv", "9": "wxyz"}
    return [''.join(p) for p in product(*(mapping[d] for d in digits))]
```

Dry run: digit '2' → a, b, c; digit '3' → d, e, f; combining each pair gives 9 total strings, first being "ad".

79. N-Queens

Problem: Place n queens on an n×n board so none attack each other; return all solutions. **Sample I/O:** n=4 → 2 solutions, e.g.

[".Q...", "...Q", "Q...", "...Q."] and its mirror

```
def solveNQueens(n):
    res = []
    cols, posDiag, negDiag = set(), set(), set()
    board = [["."]*n for _ in range(n)]
    def backtrack(r):
        if r == n:
            res.append([''.join(row) for row in board]);
        for c in range(n):
            if c in cols or (r+c) in posDiag or (r-c) in negDiag:
                continue
            cols.add(c); posDiag.add(r+c); negDiag.add(r-c)
            board[r][c] = "Q"
            backtrack(r+1)
            cols.remove(c); posDiag.remove(r+c); negDiag.remove(r-c)
            board[r][c] = "."
    backtrack(0)
    return res
```

Alternative: represent placement as a 1D array `queens[row] = col` instead of a 2D board, only building the string grid at the end when a solution is found — less memory churn during search. **Dry run:** for n=4,

valid placements avoid same column/diagonal; backtracking finds queen positions $(0, 1), (1, 3), (2, 0), (3, 2)$ as one of the 2 valid solutions.

SECTION 11: Graphs

80. Number of Islands

Problem: Count connected groups of land ('1') in a grid, 4-directionally connected. **Sample I/O:** `[["1", "1", "0"], ["1", "0", "0"], ["0", "0", "1"]] → 2`

```
def numIslands(grid):
    rows, cols = len(grid), len(grid[0])
    visit = set()
    def dfs(r, c):
        if r<0 or c<0 or r>=rows or c>=cols or grid[r][c]
        visit.add((r,c))
        for dr, dc in [(0,1),(0,-1),(1,0),(-1,0)]:
            dfs(r+dr, c+dc)
    count = 0
    for r in range(rows):
        for c in range(cols):
            if grid[r][c]=="1" and (r,c) not in visit:
                dfs(r,c); count += 1
    return count
```

Alternative (BFS with queue instead of recursive DFS, avoids recursion-depth issues on huge grids):

```
from collections import deque
def numIslands(grid):
    rows, cols = len(grid), len(grid[0])
    visit = set()
    def bfs(r, c):
        q = deque([(r,c)]); visit.add((r,c))
```

```

while q:
    row, col = q.popleft()
    for dr, dc in [(0,1), (0,-1), (1,0), (-1,0)]:
        nr, nc = row+dr, col+dc
        if 0<=nr<rows and 0<=nc<cols and grid[nr][nc] == 1 and (nr,nc) not in visit:
            visit.add((nr,nc)); q.append((nr,nc))
count = 0
for r in range(rows):
    for c in range(cols):
        if grid[r][c] == "1" and (r,c) not in visit:
            bfs(r,c); count += 1
return count

```

Dry run: DFS from (0,0) marks the connected 1s at (0,0), (0,1), (1,0) as one island; the isolated 1 at (2,2) forms a second island → total 2.

81. Max Area of Island

Problem: Return the area of the largest connected island of 1's. **Sample I/O:** grid with islands of sizes 1 and 4 → 4

```

def maxAreaOfIsland(grid):
    rows, cols = len(grid), len(grid[0])
    visit = set()
    def dfs(r, c):
        if r<0 or c<0 or r>=rows or c>=cols or grid[r][c] == 0 or (r,c) in visit:
            return 0
        visit.add((r,c))
        return 1 + dfs(r+1,c)+dfs(r-1,c)+dfs(r,c+1)+dfs(r,c-1)
    best = 0
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == 1 and (r,c) not in visit:
                best = max(best, dfs(r,c))
    return best

```

Alternative: mutate grid in place (set visited cells to 0) instead of a

separate `visit` set — saves memory, destroys input. **Dry run:** DFS from a 4-cell island sums $1+1+1+1=4$; comparing against a 1-cell island (1) → max is 4.

82. Clone Graph

Problem: Deep copy a connected undirected graph given a reference node. **Sample I/O:** graph 1-2-3-4-1 (adjacency list) → structurally identical clone

```
def cloneGraph(node):
    if not node: return None
    oldToNew = {}
    def dfs(n):
        if n in oldToNew: return oldToNew[n]
        copy = Node(n.val)
        oldToNew[n] = copy
        for nei in n.neighbors:
            copy.neighbors.append(dfs(nei))
        return copy
    return dfs(node)
```

Alternative (BFS with queue instead of recursive DFS):

```
from collections import deque
def cloneGraph(node):
    if not node: return None
    oldToNew = {node: Node(node.val)}
    q = deque([node])
    while q:
        n = q.popleft()
        for nei in n.neighbors:
            if nei not in oldToNew:
                oldToNew[nei] = Node(nei.val)
                q.append(nei)
            oldToNew[n].neighbors.append(oldToNew[nei])
    return oldToNew[node]
```

Dry run: hashmap prevents infinite recursion on cycles; visiting node 1 clones it, recurses into neighbor 2, which recurses into 3, then 4, whose neighbor 1 is already cloned (returned from map) → cycle closed correctly.

83. Islands and Treasure (Walls and Gates)

Problem: Fill each empty room with distance to nearest gate (multi-source BFS). **Sample I/O:** grid with gates(0), walls(-1), empty(INF) → empty cells filled with shortest distance to a gate

```
from collections import deque
def islandsAndTreasure(grid):
    rows, cols = len(grid), len(grid[0])
    q = deque()
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == 0: q.append((r,c))
    while q:
        r, c = q.popleft()
        for dr, dc in [(0,1), (0,-1), (1,0), (-1,0)]:
            nr, nc = r+dr, c+dc
            if 0<=nr<rows and 0<=nc<cols and grid[nr][nc] == -1:
                grid[nr][nc] = grid[r][c] + 1
                q.append((nr, nc))
```

Alternative: run DFS/BFS separately from each empty cell searching for the nearest gate — correct but $O(n \cdot m)$ per cell instead of one shared multi-source BFS, much slower. **Dry run:** all gates (value 0) start in the queue simultaneously; BFS expands outward layer by layer, so each empty cell gets filled with the minimum number of hops from *any* gate.

84. Rotting Oranges

Problem: Min minutes until no fresh orange remains (rot spreads 4-

directionally each minute). **Sample I/O:** `[[2,1,1],[1,1,0],[0,1,1]]` → 4

```
from collections import deque
def orangesRotting(grid):
    rows, cols = len(grid), len(grid[0])
    q = deque()
    fresh = 0
    for r in range(rows):
        for c in range(cols):
            if grid[r][c]==2: q.append((r,c))
            elif grid[r][c]==1: fresh += 1
    minutes = 0
    while q and fresh:
        for _ in range(len(q)):
            r, c = q.popleft()
            for dr, dc in [(0,1),(0,-1),(1,0),(-1,0)]:
                nr, nc = r+dr, c+dc
                if 0<=nr<rows and 0<=nc<cols and grid[nr][nc]==1:
                    grid[nr][nc] = 2
                    fresh -= 1
                    q.append((nr,nc))
        minutes += 1
    return minutes if fresh==0 else -1
```

Alternative: simulate minute-by-minute with a full grid scan each round instead of a queue — $O(\text{rows} \cdot \text{cols})$ per minute rather than $O(1)$ amortized per cell, simpler to write. **Dry run:** minute 1 rots adjacent fresh oranges to the initial rotten ones; this cascades outward; after 4 minutes all fresh oranges are rotten → 4.

85. Pacific Atlantic Water Flow

Problem: Find cells from which water can flow to both Pacific (top/left) and Atlantic (bottom/right) oceans. **Sample I/O:** heightmap → list of `[r,c]` coordinates satisfying both

```

def pacificAtlantic(heights):
    rows, cols = len(heights), len(heights[0])
    pac, atl = set(), set()
    def dfs(r, c, visit, prevHeight):
        if (r,c) in visit or r<0 or c<0 or r>=rows or c>=cols:
            return
        visit.add((r,c))
        for dr, dc in [(0,1), (0,-1), (1,0), (-1,0)]:
            dfs(r+dr, c+dc, visit, heights[r][c])
    for c in range(cols):
        dfs(0, c, pac, heights[0][c])
        dfs(rows-1, c, atl, heights[rows-1][c])
    for r in range(rows):
        dfs(r, 0, pac, heights[r][0])
        dfs(r, cols-1, atl, heights[r][cols-1])
    return [(r,c) for r in range(rows) for c in range(cols) if (r,c) in pac and (r,c) in atl]

```

Alternative: for each individual cell, run a full flow-simulation DFS/BFS checking whether it can reach both oceans directly — correct but $O((rows \cdot cols)^2)$, far slower than the reverse-flow-from-border approach.

Dry run: flood-fill *inward* from Pacific-adjacent border cells (uphill only) marks `pac`; same from Atlantic border marks `atl`; intersection of both sets is the answer.

86. Surrounded Regions

Problem: Capture (flip to 'X') all 'O' regions not connected to the border.

Sample I/O: `[["X", "X", "X", "X"], ["X", "O", "O", "X"], ["X", "X", "O", "X"], ["X", "O", "X", "X"]]` → middle O's captured, border-connected one stays

```

def solve(board):
    rows, cols = len(board), len(board[0])
    def dfs(r, c):
        if r<0 or c<0 or r>=rows or c>=cols or board[r][c] != 'O':
            return
        board[r][c] = '#'
        for dr, dc in [(0,1), (0,-1), (1,0), (-1,0)]:
            dfs(r+dr, c+dc)
    for r in range(rows):
        dfs(r, 0)
        dfs(r, cols-1)
    for c in range(1, cols-1):
        dfs(0, c)
        dfs(rows-1, c)
    for r in range(1, rows-1):
        for c in range(1, cols-1):
            if board[r][c] == '#':
                board[r][c] = 'X'

```

```

        dfs(r, 0); dfs(r, cols-1)
    for c in range(cols):
        dfs(0, c); dfs(rows-1, c)
    for r in range(rows):
        for c in range(cols):
            if board[r][c] == "O": board[r][c] = "X"
            elif board[r][c] == "#": board[r][c] = "O"

```

Alternative: use Union-Find, uniting every 'O' with its neighbors and with a virtual "border" node; any 'O' not united with the border node gets flipped — same result, different data structure. **Dry run:** border-connected O's get marked '#' (protected); remaining untouched Os (fully enclosed) become X; then restore # back to O.

87. Course Schedule

Problem: Determine if all courses can be finished given prerequisite pairs (i.e., no cycle). **Sample I/O:** numCourses=2, prerequisites=[[1,0]] → True

```

def canFinish(numCourses, prerequisites):
    graph = {i: [] for i in range(numCourses)}
    for course, pre in prerequisites: graph[course].append(pre)
    state = {} # 0=visiting, 1=done
    def dfs(course):
        if course in state: return state[course] == 1
        state[course] = 0
        for pre in graph[course]:
            if not dfs(pre): return False
        state[course] = 1
        return True
    return all(dfs(c) for c in range(numCourses))

```

Alternative (Kahn's algorithm, BFS topological sort using in-degrees):

```

from collections import deque

```

```
def canFinish(numCourses, prerequisites):
    graph = {i: [] for i in range(numCourses)}
    indegree = [0]*numCourses
    for course, pre in prerequisites:
        graph[pre].append(course)
        indegree[course] += 1
    q = deque([c for c in range(numCourses) if indegree[c] == 0])
    visited = 0
    while q:
        c = q.popleft(); visited += 1
        for nxt in graph[c]:
            indegree[nxt] -= 1
            if indegree[nxt]==0: q.append(nxt)
    return visited == numCourses
```

Dry run: course 1 requires 0; DFS on 1 visits 0 (no prereqs, marked done), then 1 marked done → no cycle → True.

88. Course Schedule II

Problem: Return a valid course order to finish all courses, or [] if impossible. **Sample I/O:** numCourses=4, prerequisites=[[1,0],[2,0],[3,1],[3,2]] → [0,1,2,3] (one valid order)

```
from collections import deque
def findOrder(numCourses, prerequisites):
    graph = {i: [] for i in range(numCourses)}
    indegree = [0]*numCourses
    for course, pre in prerequisites:
        graph[pre].append(course)
        indegree[course] += 1
    q = deque([c for c in range(numCourses) if indegree[c] == 0])
    order = []
    while q:
        c = q.popleft(); order.append(c)
        for nxt in graph[c]:
            indegree[nxt] -= 1
            if indegree[nxt] == 0: q.append(nxt)
```

```

        if indegree[nxt]==0: q.append(nxt)
    return order if len(order)==numCourses else []

```

Alternative (DFS-based postorder topological sort, reverse the finish order): run DFS marking visiting/done states like Course Schedule, appending each course to a list when fully processed, then reverse that list. **Dry run:** indegree [0, 1, 1, 2]; queue starts with 0; processing 0 decrements 1, 2 to indegree 0, adds both; processing them decrements 3 to 0 → order [0, 1, 2, 3].

89. Graph Valid Tree

Problem: Given n nodes and edges, determine if they form a valid tree (connected, no cycles). **Sample I/O:** n=5, edges=[[0,1],[0,2],[0,3],[1,4]] → True

```

def validTree(n, edges):
    if len(edges) != n-1: return False
    graph = {i: [] for i in range(n)}
    for a, b in edges:
        graph[a].append(b); graph[b].append(a)
    visit = set()
    def dfs(node, parent):
        visit.add(node)
        for nei in graph[node]:
            if nei == parent: continue
            if nei in visit: return False
            if not dfs(nei, node): return False
        return True
    return dfs(0, -1) and len(visit) == n

```

Alternative (Union-Find: valid tree iff exactly n-1 edges and no union operation ever joins two already-connected nodes):

```

def validTree(n, edges):
    if len(edges) != n-1: return False
    parent = list(range(n))

```

```

def find(x):
    while parent[x] != x:
        parent[x] = parent[parent[x]]
        x = parent[x]
    return x
for a, b in edges:
    ra, rb = find(a), find(b)
    if ra == rb: return False
    parent[ra] = rb
return True

```

Dry run: edge count $4 \leq 5-1$ passes prerequisite check; DFS from node 0 visits all 5 nodes without revisiting any non-parent node $\rightarrow$ connected and acyclic $\rightarrow$ True.

90. Number of Connected Components in an Undirected Graph

Problem: Count connected components given n nodes and edge list.

Sample I/O: $n=5$, $edges=[[0,1],[1,2],[3,4]] \rightarrow 2$

```

def countComponents(n, edges):
    parent = list(range(n))
    rank = [1]*n
    def find(x):
        while parent[x] != x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x
    def union(a, b):
        ra, rb = find(a), find(b)
        if ra == rb: return 0
        if rank[ra] < rank[rb]: ra, rb = rb, ra
        parent[rb] = ra
        rank[ra] += rank[rb]
        return 1
    components = n

```

```

for a, b in edges:
    components -= union(a, b)
return components

```

Alternative (DFS/BFS from each unvisited node, counting how many sweeps are needed):

```

def countComponents(n, edges):
    graph = {i: [] for i in range(n)}
    for a, b in edges:
        graph[a].append(b); graph[b].append(a)
    visit = set()
    def dfs(node):
        for nei in graph[node]:
            if nei not in visit:
                visit.add(nei); dfs(nei)
    count = 0
    for node in range(n):
        if node not in visit:
            visit.add(node); dfs(node); count += 1
    return count

```

Dry run: union(0,1), union(1,2) merge {0, 1, 2} into one set; union(3,4) merges {3, 4}; two disjoint sets remain → 2.

91. Redundant Connection

Problem: Given a tree with one extra edge added (creating exactly one cycle), find that redundant edge. **Sample I/O:** edges=[[1, 2], [1, 3], [2, 3]] → [2, 3]

```

def findRedundantConnection(edges):
    n = len(edges)
    parent = list(range(n+1))
    def find(x):
        while parent[x] != x:
            parent[x] = parent[parent[x]]

```

```

        x = parent[x]
    return x
for a, b in edges:
    ra, rb = find(a), find(b)
    if ra == rb: return [a, b]
    parent[ra] = rb
return []

```

Alternative (DFS-based cycle detection: for each edge, check if a path already connects its endpoints before adding it): build the graph incrementally, and before adding each edge run a DFS/BFS to see if the two endpoints are already reachable from each other. **Dry run:** union(1,2) merges roots; union(1,3) merges again; union(2,3): find(2) and find(3) are already the same root → this edge closes a cycle → return [2,3].

92. Word Ladder

Problem: Shortest transformation sequence length from `beginWord` to `endWord`, changing one letter at a time via words in `wordList`. **Sample**

I/O: `beginWord="hit", endWord="cog", wordList=["hot", "dot", "dog", "lot", "log", "cog"]` → 5

```

from collections import deque
def ladderLength(beginWord, endWord, wordList):
    wordSet = set(wordList)
    if endWord not in wordSet: return 0
    q = deque([(beginWord, 1)])
    while q:
        word, steps = q.popleft()
        if word == endWord: return steps
        for i in range(len(word)):
            for c in 'abcdefghijklmnopqrstuvwxyz':
                nxt = word[:i]+c+word[i+1:]
                if nxt in wordSet:
                    wordSet.remove(nxt)
                    q.append((nxt, steps+1))

```

```
return 0
```

Alternative (bidirectional BFS, expanding from both begin and end simultaneously — much faster on large word lists): maintain two frontiers, always expanding the smaller one, and stop when they meet.

Dry run: BFS layers: hit→hot→dot/lot→dog/log→cog; reaching cog at step 5 (hit=1, hot=2, dot=3, dog=4, cog=5) → 5.

SECTION 12: Advanced Graphs (first 8)

93. Reconstruct Itinerary

Problem: Given airline tickets `[from, to]`, reconstruct itinerary starting from "JFK" that uses all tickets, lexicographically smallest if multiple exist.

Sample I/O: tickets = `[["MUC", "LHR"], ["JFK", "MUC"], ["SFO", "SJC"], ["LHR", "SFO"]]` → `["JFK", "MUC", "LHR", "SFO", "SJC"]`

```
from collections import defaultdict
def findItinerary(tickets):
    graph = defaultdict(list)
    for a, b in sorted(tickets, reverse=True):
        graph[a].append(b)
    route = []
    def dfs(node):
        while graph[node]:
            dfs(graph[node].pop())
        route.append(node)
    dfs("JFK")
    return route[::-1]
```

Alternative: build adjacency sorted ascending, do standard DFS with backtracking (try each destination, undo if it doesn't lead to using all tickets) — conceptually simpler but can be slower without Hierholzer's trick. **Dry run:** Hierholzer's algorithm processes Eulerian path by always

going deepest first and appending to route only when a node has no more outgoing tickets, then reversing the finish order →
JFK, MUC, LHR, SFO, SJC.

94. Min Cost to Connect All Points

Problem: Connect all points with min total Manhattan-distance edges (Minimum Spanning Tree). **Sample I/O:** `points=[[0,0],[2,2],[3,10],[5,2],[7,0]]` → 20

```
import heapq
def minCostConnectPoints(points):
    n = len(points)
    visited = set()
    minHeap = [(0, 0)] # (cost, point index)
    total = 0
    while len(visited) < n:
        cost, i = heapq.heappop(minHeap)
        if i in visited: continue
        visited.add(i)
        total += cost
        for j in range(n):
            if j not in visited:
                dist = abs(points[i][0]-points[j][0]) + abs(points[i][1]-points[j][1])
                heapq.heappush(minHeap, (dist, j))
    return total
```

Alternative (Kruskal's algorithm with Union-Find, sort all edges by weight):

```
def minCostConnectPoints(points):
    n = len(points)
    edges = []
    for i in range(n):
        for j in range(i+1, n):
            dist = abs(points[i][0]-points[j][0]) + abs(points[i][1]-points[j][1])
            edges.append((dist, i, j))
```

```

edges.sort()
parent = list(range(n))
def find(x):
    while parent[x] != x: parent[x] = parent[parent[x]]
    return x
total, count = 0, 0
for dist, a, b in edges:
    ra, rb = find(a), find(b)
    if ra != rb:
        parent[ra] = rb; total += dist; count += 1
        if count == n-1: break
return total

```

Dry run: Prim's algorithm grows the MST greedily from point 0, always picking the cheapest edge to an unvisited point — total accumulated cost across all 5 points is 20.

95. Network Delay Time

Problem: Min time for a signal to reach all n nodes from source k (Dijkstra's algorithm). **Sample I/O:** times=[[2,1,1],[2,3,1],[3,4,1]], $n=4$, $k=2 \rightarrow 2$

```

import heapq
from collections import defaultdict
def networkDelayTime(times, n, k):
    graph = defaultdict(list)
    for u, v, w in times: graph[u].append((v, w))
    minHeap = [(0, k)]
    visited = {}
    while minHeap:
        t, node = heapq.heappop(minHeap)
        if node in visited: continue
        visited[node] = t
        for nei, w in graph[node]:
            if nei not in visited:
                heapq.heappush(minHeap, (t+w, nei))

```

```
return max(visited.values()) if len(visited) == n else
```

Alternative (Bellman-Ford, handles negative weights though not needed here):

```
def networkDelayTime(times, n, k):
    dist = {i: float('inf') for i in range(1, n+1)}
    dist[k] = 0
    for _ in range(n-1):
        for u, v, w in times:
            if dist[u] + w < dist[v]: dist[v] = dist[u] + w
    result = max(dist.values())
    return result if result != float('inf') else -1
```

Dry run: Dijkstra pops (0, 2), relaxes to (1, 1) and (1, 3); pops (1, 1) (node 1 done); pops (1, 3), relaxes to (2, 4); final max distance among reached nodes is 2.

96. Swim in Rising Water

Problem: Min time to swim from top-left to bottom-right where water level rises; can move to a cell once water level $\geq$ its elevation. **Sample I/**

O: grid=[[0, 2], [1, 3]] $\rightarrow$ 3

```
import heapq
def swimInWater(grid):
    n = len(grid)
    visit = {(0,0)}
    minHeap = [(grid[0][0], 0, 0)]
    while minHeap:
        t, r, c = heapq.heappop(minHeap)
        if (r,c) == (n-1,n-1): return t
        for dr, dc in [(0,1), (0,-1), (1,0), (-1,0)]:
            nr, nc = r+dr, c+dc
            if 0<=nr<n and 0<=nc<n and (nr,nc) not in visit:
                visit.add((nr,nc))
                heapq.heappush(minHeap, (max(t, grid[nr][nc]), nr, nc))
```

Alternative (binary search on time + BFS/DFS feasibility check at each candidate time):

```
from collections import deque
def swimInWater(grid):
    n = len(grid)
    def canReach(t):
        if grid[0][0] > t: return False
        visit = {(0,0)}
        q = deque([(0,0)])
        while q:
            r, c = q.popleft()
            if (r,c) == (n-1,n-1): return True
            for dr, dc in [(0,1), (0,-1), (1,0), (-1,0)]:
                nr, nc = r+dr, c+dc
                if 0<=nr<n and 0<=nc<n and (nr,nc) not in visit:
                    visit.add((nr,nc)); q.append((nr,nc))
        return False
    lo, hi = grid[0][0], n*n-1
    while lo < hi:
        mid = (lo+hi)//2
        if canReach(mid): hi = mid
        else: lo = mid+1
    return lo
```

Dry run: Dijkstra-like search always expands the path with the lowest "max elevation so far"; reaching (1,1) requires passing through elevation 3 at minimum → answer 3.

97. Alien Dictionary

Problem: Given words sorted according to an alien language's rules, derive a valid character ordering. **Sample I/O:**

words=["wrt", "wrf", "er", "ett", "rftt"] → "wertf"

```
def alienOrder(words):
    graph = {c: set() for w in words for c in w}
```

```

for w1, w2 in zip(words, words[1:]):
    minLen = min(len(w1), len(w2))
    if w1[:minLen] == w2[:minLen] and len(w1) > len(w2):
        for c1, c2 in zip(w1, w2):
            if c1 != c2:
                graph[c1].add(c2)
                break
state = {} # 0=visiting, 1=done
res = []
def dfs(c):
    if c in state: return state[c] == 1
    state[c] = 0
    for nei in graph[c]:
        if not dfs(nei): return False
    state[c] = 1
    res.append(c)
    return True
for c in graph:
    if not dfs(c): return ""
return ''.join(res[::-1])

```

Alternative (Kahn's BFS topological sort using in-degree counts, same edge-building step): build the same graph, compute in-degrees, then repeatedly pop zero-in-degree characters into the result — avoids recursion. **Dry run:** comparing adjacent words gives edges $w \rightarrow e$, $r \rightarrow t$, $t \rightarrow f$, $e \rightarrow r$; topological sort of these edges yields order $w, e, r, t, f \rightarrow \text{"wertf"}$.

98. Cheapest Flights Within K Stops

Problem: Cheapest price from `src` to `dst` using at most `k` stops (Bellman-Ford variant). **Sample I/O:** `n=4, flights=[[0,1,100], [1,2,100], [2,0,100], [1,3,600], [2,3,200]]`, `src=0`, `dst=3`, `k=1` $\rightarrow$ 700

```

def findCheapestPrice(n, flights, src, dst, k):
    prices = [float('inf')] * n

```

```

prices[src] = 0
for _ in range(k+1):
    tmpPrices = prices[:]
    for s, d, w in flights:
        if prices[s] != float('inf') and prices[s] +
            tmpPrices[d] = prices[s] + w
    prices = tmpPrices
return prices[dst] if prices[dst] != float('inf') else

```

Alternative (Dijkstra variant tracking stops used, via priority queue of (cost, node, stopsUsed)):

```

import heapq
from collections import defaultdict
def findCheapestPrice(n, flights, src, dst, k):
    graph = defaultdict(list)
    for s, d, w in flights: graph[s].append((d, w))
    minHeap = [(0, src, 0)]
    while minHeap:
        cost, node, stops = heapq.heappop(minHeap)
        if node == dst: return cost
        if stops > k:
            continue
        for nei, w in graph[node]:
            heapq.heappush(minHeap, (cost+w, nei, stops+1))
    return -1

```

Dry run: round 1 (0 stops) relaxes 0→1 to cost 100; round 2 (≤ 1 stop) relaxes 1→3 to 100+600=700; $k=1$ allows exactly one intermediate stop → 700.

99. Course Schedule (Advanced revisit — see #87 for base version); this slot covers Graph Valid Tree-adjacent problem: Redundant Connection II (directed graph variant)

Problem: A rooted tree has one extra directed edge added; find the

edge to remove to restore a valid rooted tree. **Sample I/O:**

edges=[[1,2],[1,3],[2,3]] → [2,3]

```
def findRedundantDirectedConnection(edges):
    n = len(edges)
    parent = [0]*(n+1)
    candidate1 = candidate2 = None
    for i, (u, v) in enumerate(edges):
        if parent[v] != 0:
            candidate1 = [parent[v], v]
            candidate2 = [u, v]
            edges[i] = [0, 0] # neutralize this edge temporarily
        else:
            parent[v] = u
    uf = list(range(n+1))
    def find(x):
        while uf[x] != x: uf[x] = uf[uf[x]]; x = uf[x]
        return x
    for u, v in edges:
        if u == 0 and v == 0: continue
        ru, rv = find(u), find(v)
        if ru == rv:
            return candidate1 if candidate1 else [u, v]
        uf[ru] = rv
    return candidate2
```

Alternative: simpler two-pass approach — first detect if any node has two parents (in-degree 2), then test removing each of those two candidate edges to see which one leaves a valid tree (via cycle-detection union-find) — same idea, more explicit passes. **Dry run:** node 3 has two parents (1 and 2) → candidates recorded; after neutralizing the second occurrence, union-find still finds a cycle among the remaining edges → candidate1 ([2,3], the one that both caused the two-parent conflict and closes a cycle) is returned.

100. Number of Islands II (*Union-Find variant, tests dynamic connectivity*)

Problem: Given a grid initially all water, process a stream of land additions; after each addition, return the current number of islands.

Sample I/O: `m=3, n=3, positions=[[0,0],[0,1],[1,2],[1,0]]` → `[1,1,2,3]`

```
def numIslands2(m, n, positions):
    parent = {}
    rank = {}
    def find(x):
        while parent[x] != x: parent[x] = parent[parent[x]]
        return x
    def union(a, b):
        ra, rb = find(a), find(b)
        if ra == rb: return 0
        if rank[ra] < rank[rb]: ra, rb = rb, ra
        parent[rb] = ra
        rank[ra] += rank[rb]
        return 1
    grid = [[0]*n for _ in range(m)]
    islands = 0
    res = []
    for r, c in positions:
        if grid[r][c] == 1:
            res.append(islands); continue
        grid[r][c] = 1
        parent[(r,c)] = (r,c)
        rank[(r,c)] = 1
        islands += 1
        for dr, dc in [(0,1),(0,-1),(1,0),(-1,0)]:
            nr, nc = r+dr, c+dc
            if 0<=nr<m and 0<=nc<n and grid[nr][nc]==1:
                islands -= union((r,c), (nr,nc))
        res.append(islands)
    return res
```

Alternative: re-run a full BFS/DFS island-count (like problem #80) from scratch after every single addition — correct but $O(k \cdot m \cdot n)$ instead of near- $O(k \cdot \alpha(n))$, far slower for long position streams. **Dry run:** add `(0,0)` → `islands=1`; add `(0,1)` → adjacent, unites, islands stays 1; add

$(1, 2) \rightarrow$ isolated, islands=2; add $(1, 0) \rightarrow$ new land makes 3 total then unions with $(0, 0)$, bringing it down by 1 $\rightarrow$ islands=3 overall, matching $[1, 1, 2, 3]$.

Summary

That completes the first 100 problems following the NeetCode 250 roadmap order:

1. Arrays & Hashing (1–9)
2. Two Pointers (10–14)
3. Sliding Window (15–20)
4. Stack (21–27)
5. Binary Search (28–34)
6. Linked List (35–45)
7. Trees (46–60)
8. Tries (61–63)
9. Heap / Priority Queue (64–70)
10. Backtracking (71–79)
11. Graphs (80–92)
12. Advanced Graphs (93–100)

Next 150 (if you want to continue): remaining Advanced Graphs, 1-D DP, 2-D DP, Greedy, Intervals, Math & Geometry, Bit Manipulation.